

Введение в компьютерное зрение

В презентации используются материалы лекций Антона Конушина по предмету «Компьютерное зрение» на Факультете вычислительной математики и кибернетики МГУ

источник www.lektorium.tv

Задача компьютерного зрения

- «To see means to know what is where by looking»
 - *David Marr, **Vision**, 1982*
- «Тест Тьюринга» - компьютер должен ответить на любой вопрос об изображении, на который может ответить человек
- Что это в действительности обозначает?
 - Зрение - источник семантической информации о мире
 - Зрение - источник метрической информации о трехмерном мире

Семантическая информация



Slide credit: Fei-Fei, Fergus & Torralba

Классификация сцены

- вне помещения
- город
- уличное движение
- Пекин, Китай
- Пл. Тяньаньмэнь



Slide credit: Fei-Fei, Fergus & Torralba

Выделение объектов



Выделение границ объектов



Признаки объектов



Почему зрение это сложно

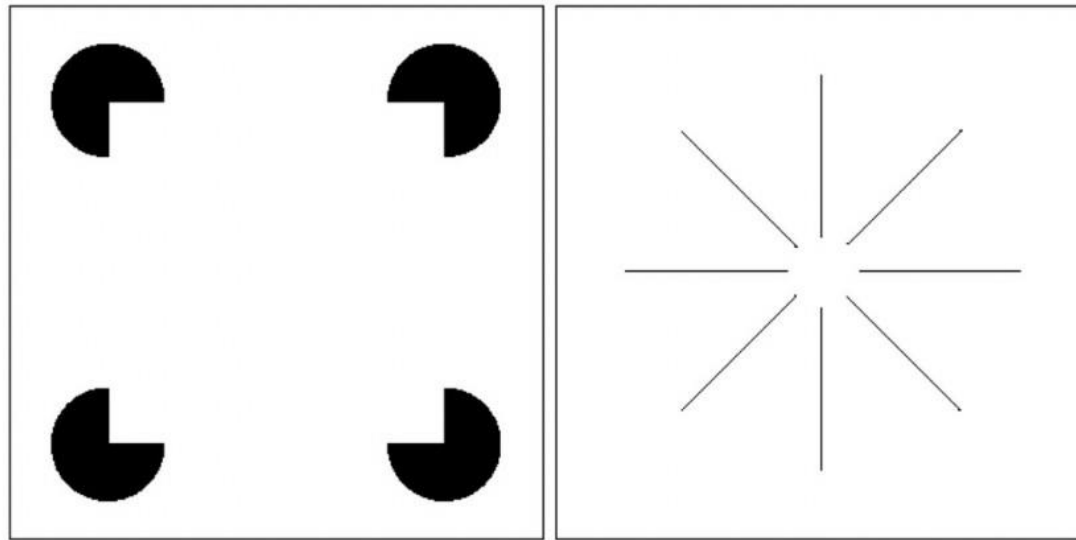
- ✓ 25+% мозга человека отвечает за зрение
- ✓ Неоднозначность интерпретации
- ✓ «ИИ-полная» задача – решение задачи зрения на уровне человека равносильно решению задачи искусственного интеллекта



Наш мозг склонен видеть то, что мы хотим видеть, а как научить такому же компьютер — открытый вопрос.

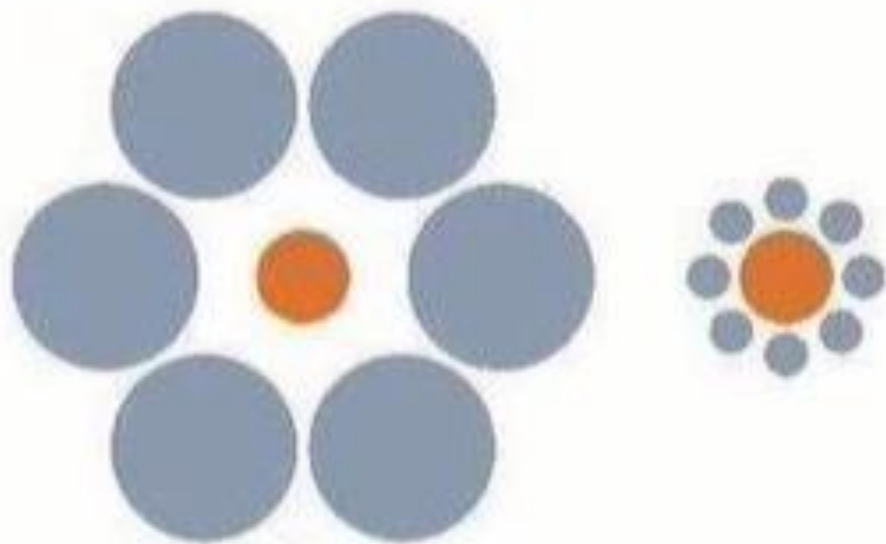
Несколько фактов о нашем зрительном восприятии

- Мы ищем знакомые образы, мозг «достраивает» картинку

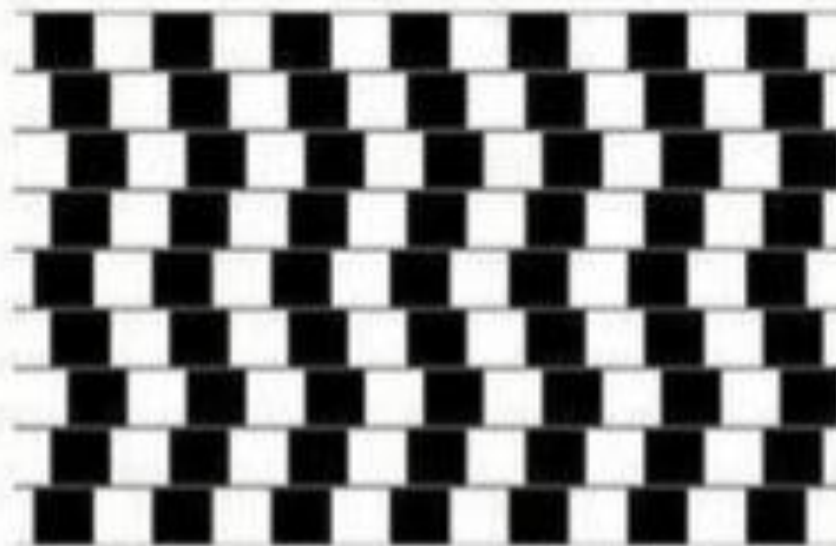


Известные оптические иллюзии

Оптические иллюзии

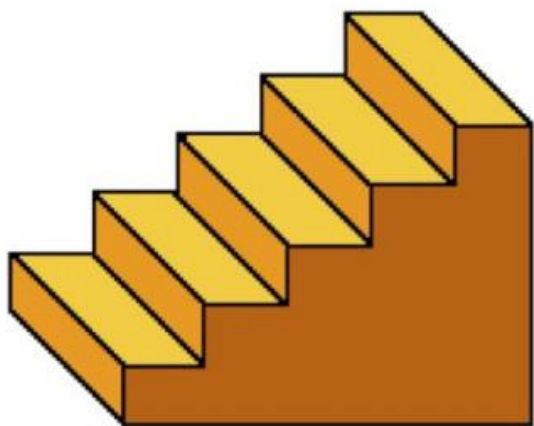


на второй картинке кружок в центре кажется больше, чем на первой!



горизонтальные линии кажутся косыми (наклонными)

От того, где мы выросли, зависит, как мы «видим» мир вокруг нас

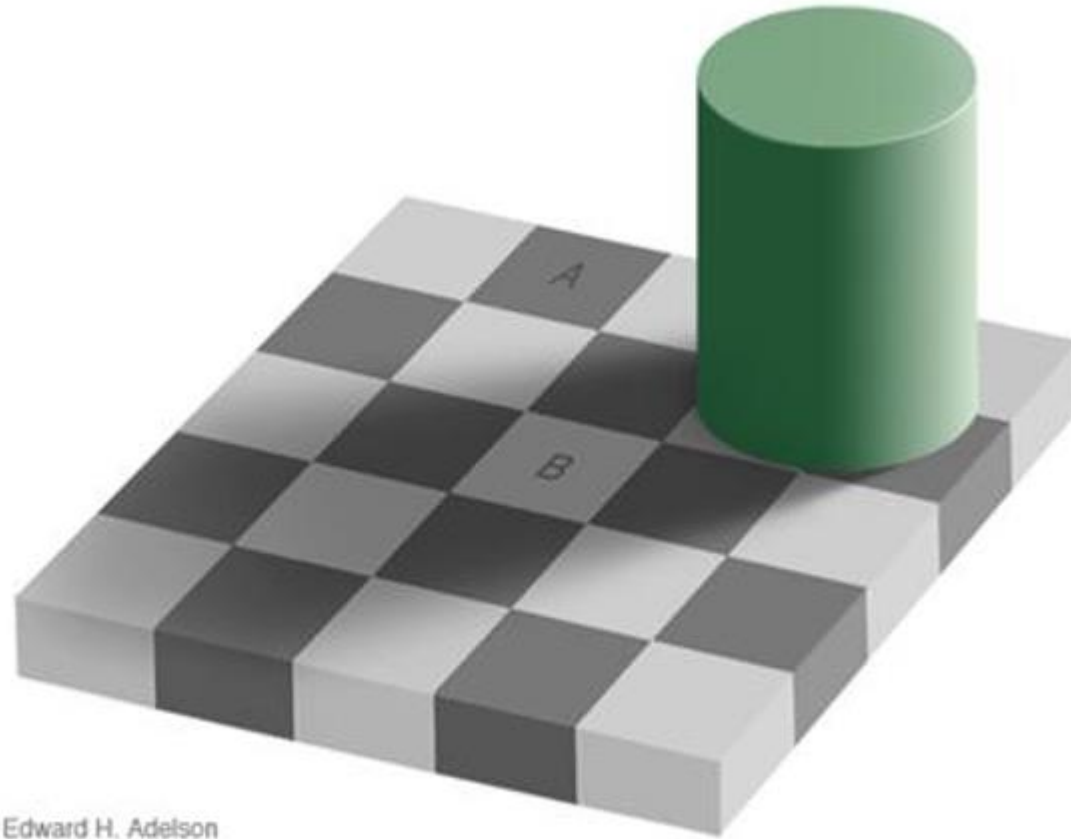


Ступеньки вверх или вниз?

Хроматическая адаптация

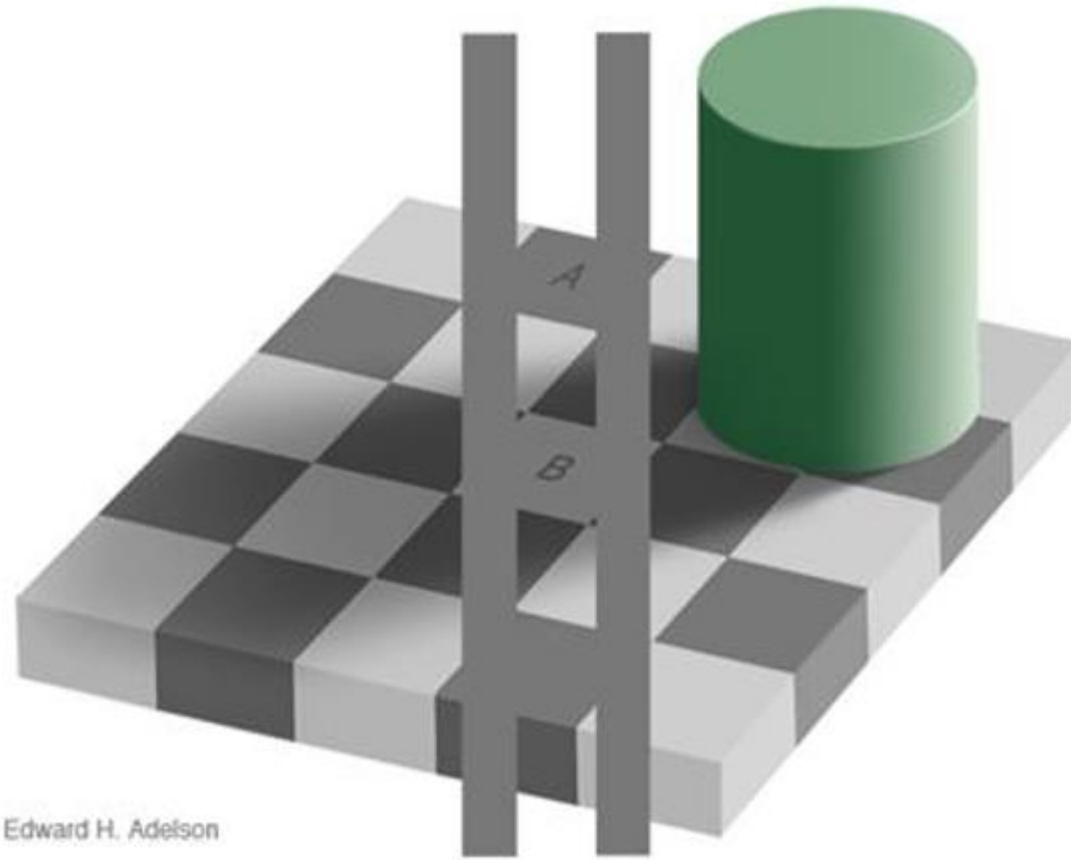


Постоянство яркости



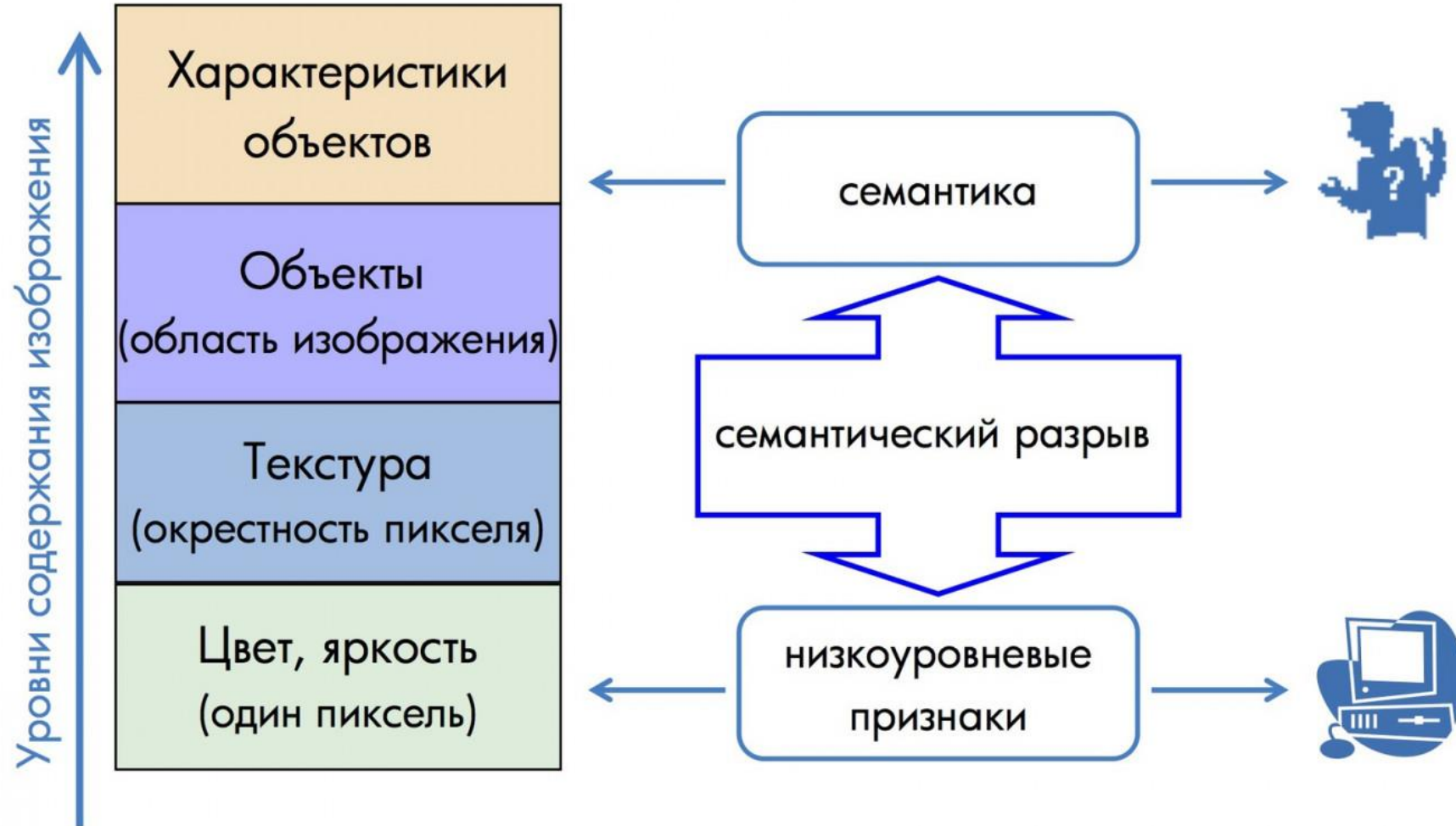
Edward H. Adelson

Постоянство яркости



Edward H. Adelson

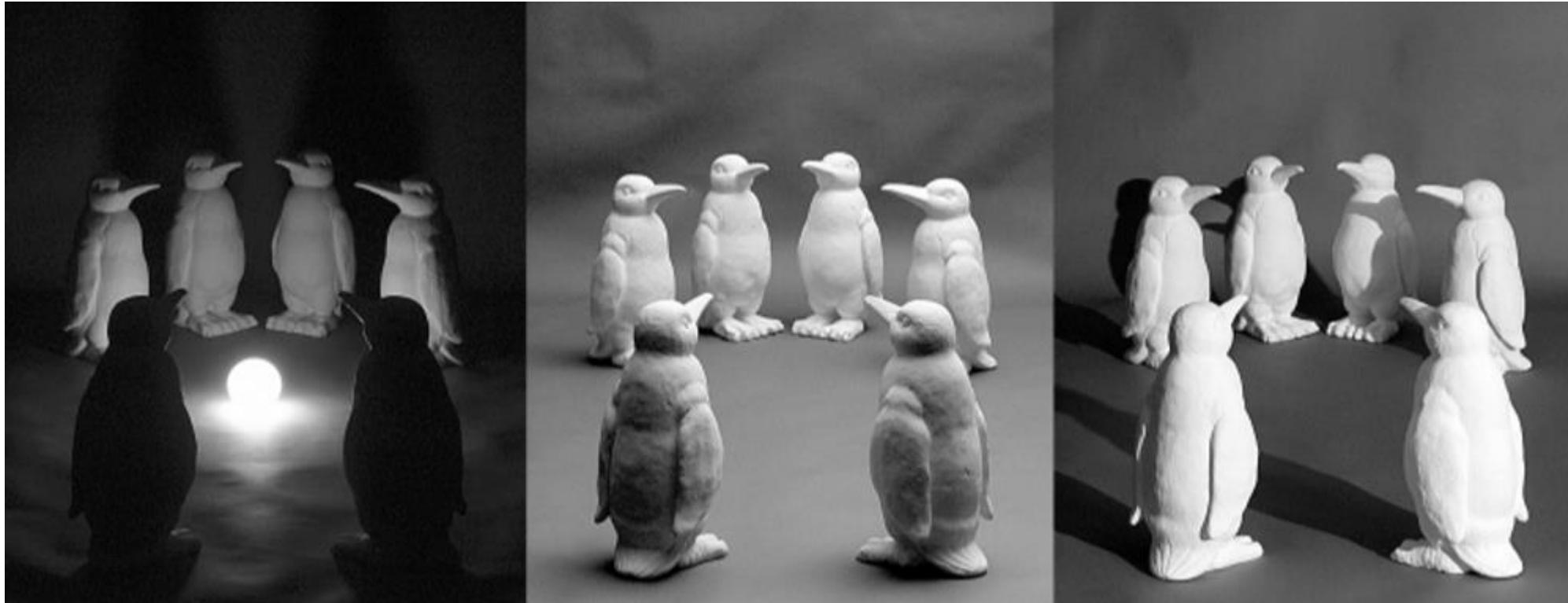
Семантический разрыв



Точка наблюдения



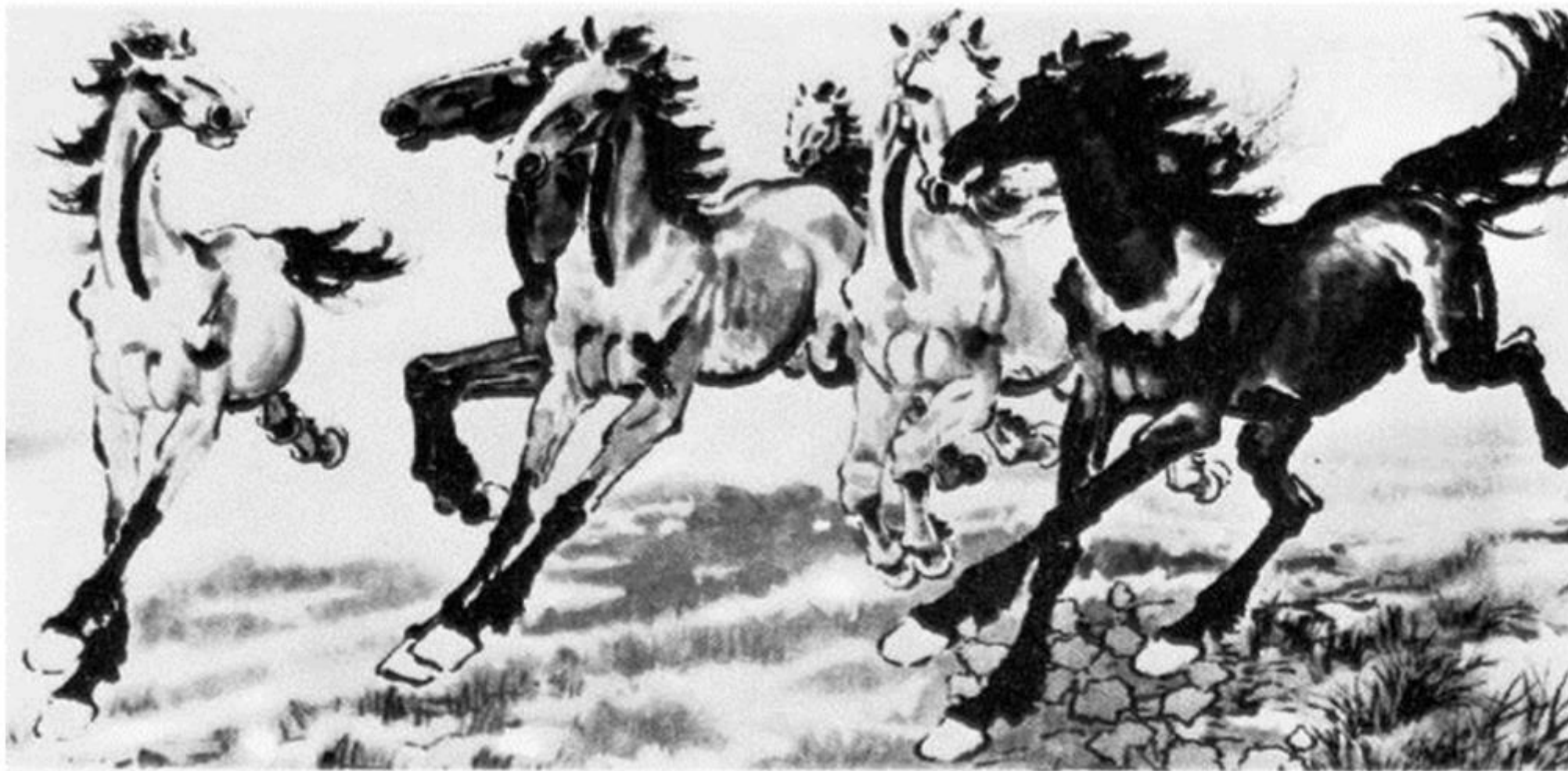
Освещенность



Масштаб



Деформация



Xu, Beihong 1943

Перекрытие

Magritte, 1957



Маскировка



Движение



Внутриклассовая изменчивость





Illumination



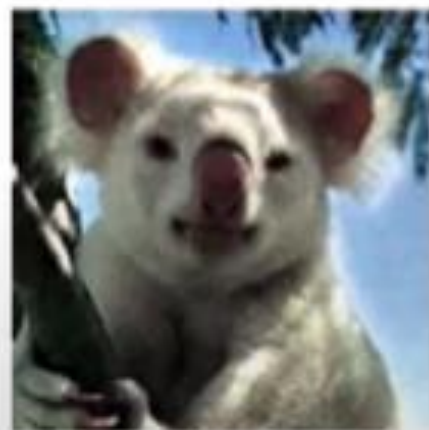
Object pose



Clutter



Occlusions



**Intra-class
appearance**



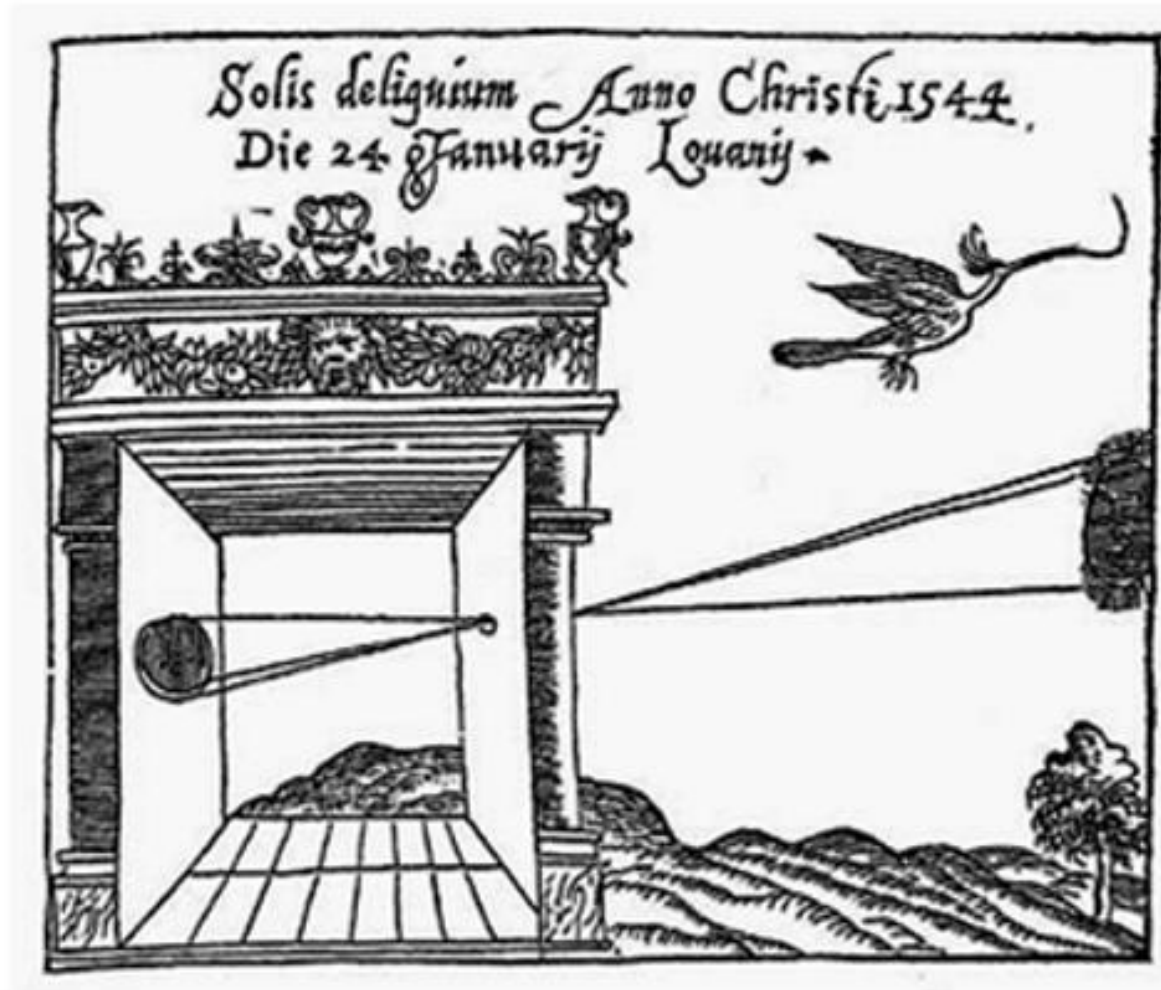
Viewpoint

Выводы: зрение изначально нечеткая задача

- Разные 3D сцены дают одно и то же 2D изображение
- Необходимы априорные знания о структуре и свойствах мира
- Нам нужно сопоставлять наблюдения и априорные знания



История: Камера-обскура



- Принцип был известен еще Аристотелю (384-322 до Н.Э.)
- Помогала художникам: описана Леонардо да Винчи (1452-1519)

История: 1525



Figure 3. Albrecht Duerer

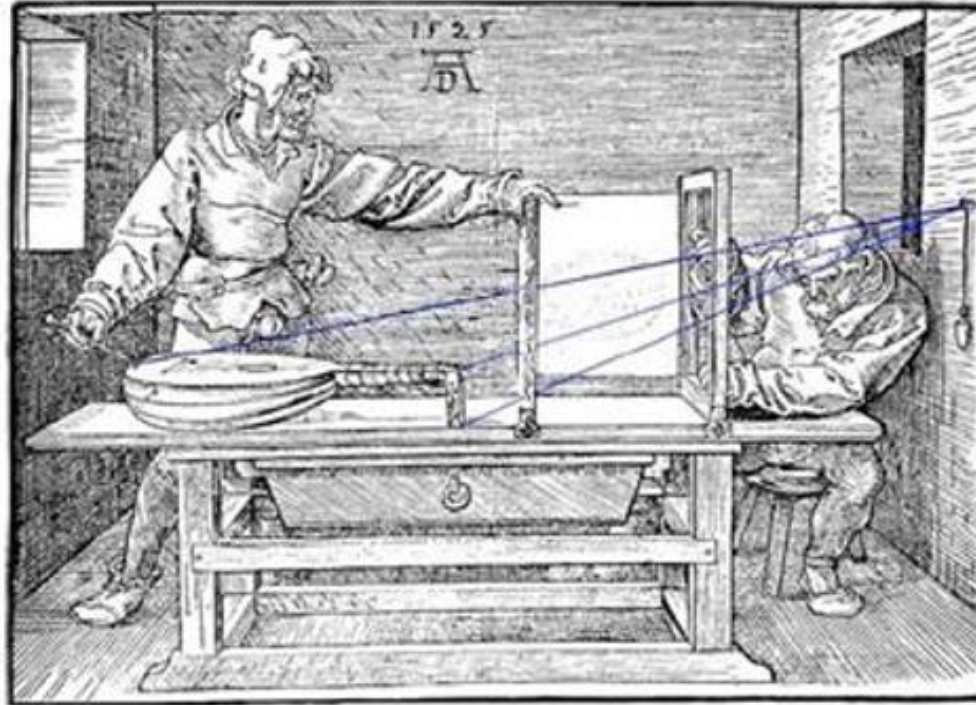
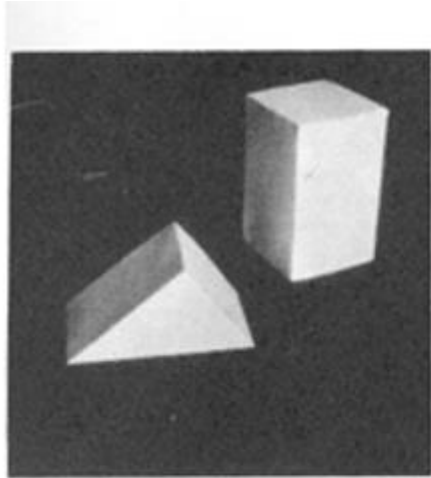


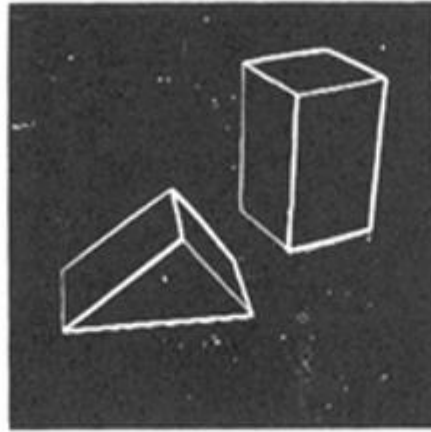
Figure 4. Woodcut by Duerer *The Draughtsman And The Lute* from his book: *'Treatise on Mensuration with the Compass and Ruler in Lines, Planes, and Whole Bodies'* (1525)



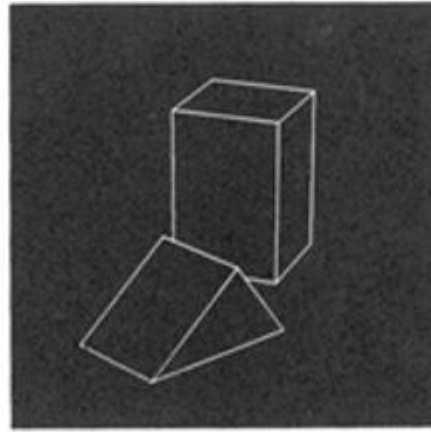
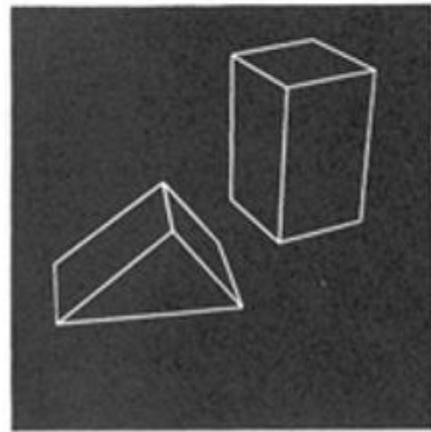
История: Положено начало компьютерному зрению



(a) Original picture.



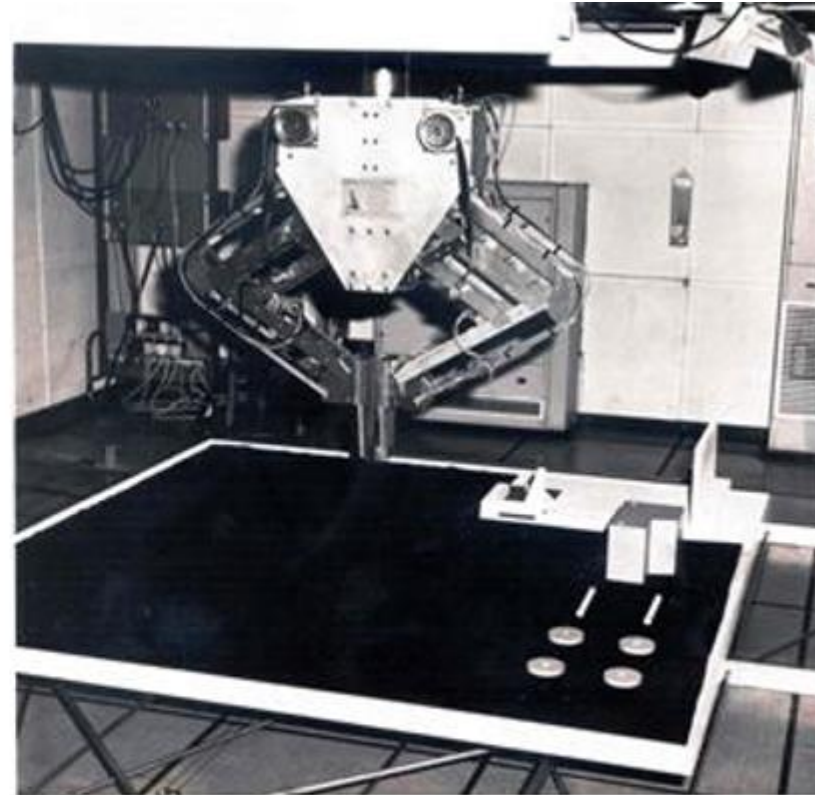
(b) Differentiated picture.



L. G. Roberts, *Machine Perception of Three Dimensional Solids*,
Ph.D. thesis, MIT Department of
Electrical Engineering, 1960

История: Freddy II, 1973

- Университет Эдинбурга
- Один из первых роботов с системой машинного зрения
- 5 степеней свободы
- Умеет собирать машинки из кубиков, разбросанных по столу
- 384Kб RAM в управляющем компьютере



Обнаружение лица и улыбки

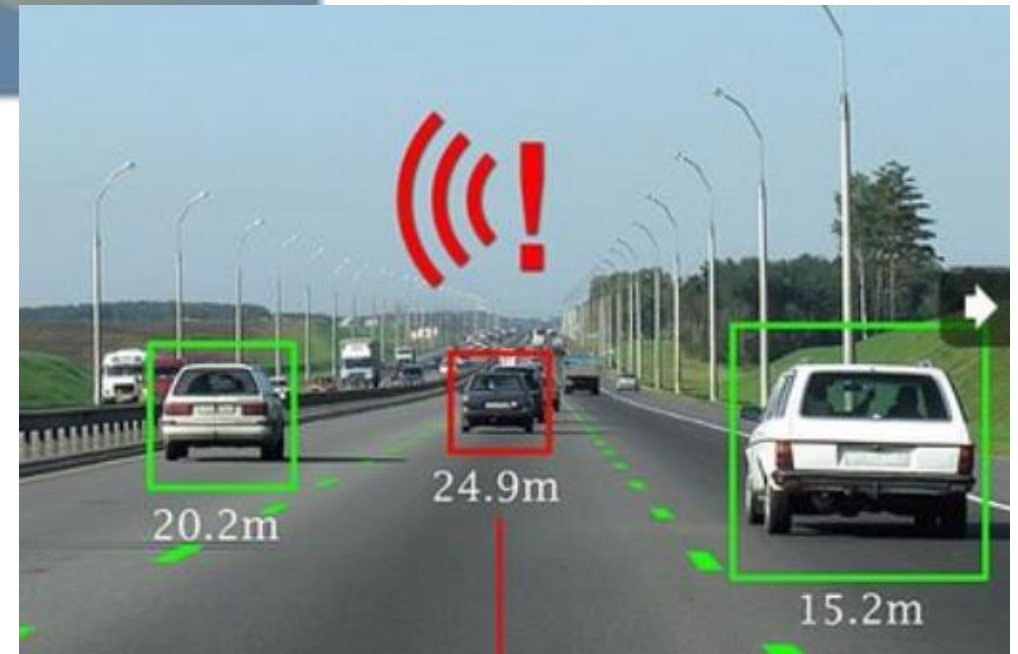
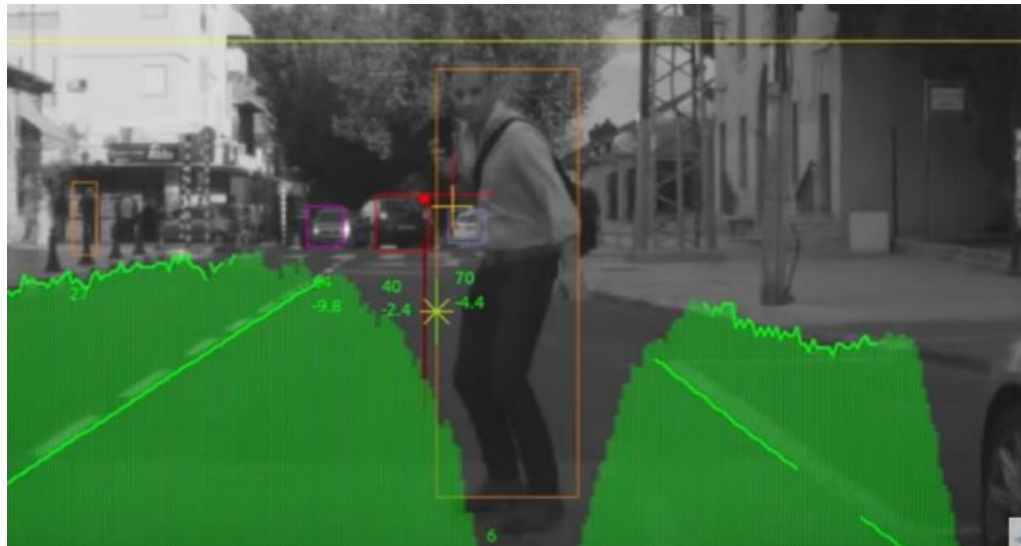
The Smile Shutter flow

Imagine a camera smart enough to catch every smile! In Smile Shutter Mode, your Cyber-shot® camera can automatically trip the shutter at just the right instant to catch the perfect expression.



Умные машины

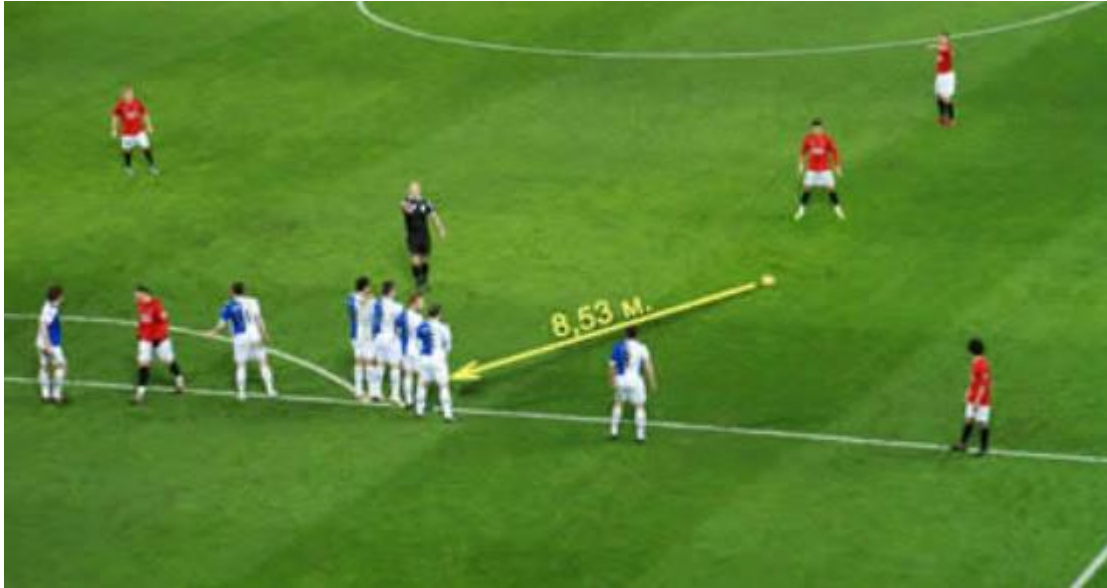
К 2010: 70% производителей машин



3D модели и захват движения



Спортивные соревнования



Зрение в космосе

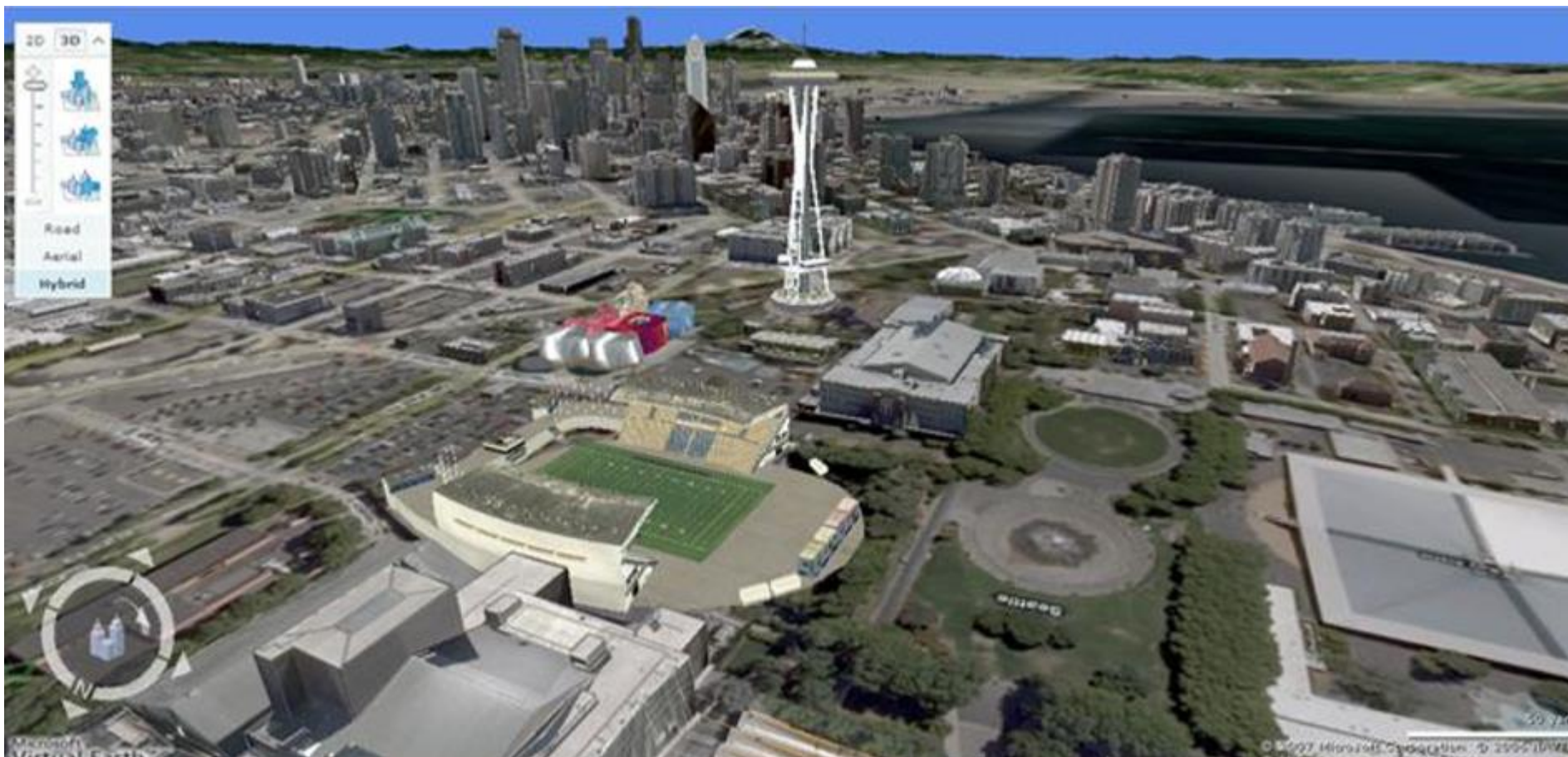


[NASA'S Mars Exploration Rover Spirit.](#)

Системы зрения использовались для:

- Склейка панорам
- 3D моделирование местности
- Поиск препятствий, определение местоположения

Трехмерные карты



Изображение

Изображение *оптическое* – картина, получаемая в результате прохождения через оптическую систему лучей, распространяющихся от объекта, и воспроизводящая его контуры и детали.

Физический энциклопедический словарь.

Функция интенсивности (яркости) канала,
заданная на 2х мерной сетке (матрице)

$$I = g(x, y), \{x \in [x_0, x_1], y \in [y_0, y_1]\}$$

Используется дискретное представление

$$I = g(i, j), \{i = \overline{1, n}, j \in \overline{1, m}\}$$

Обработка изображений

Семейство методов и задач, где входной и выходной информацией являются изображения.

Цели

1. Улучшение изображения **для восприятия человеком**
 - чтобы стало «лучше» с субъективной точки зрения человека
2. Улучшение изображения **для восприятия компьютером**
 - упрощение последующего распознавания
3. **Развлечение** (спецэффекты)
 - получить эстетическое удовольствие от красивого эффекта

Обработка изображений

Можно разделить на:

- **задачи фильтрации** (удаление шумов, повышение чёткости, осветление тёмных участков);
- **задачи анализа** (расчёт градиентов, поиск особых точек и границ объектов).

В процессе создания систем компьютерного зрения **фильтрация** изображения обычно предшествует извлечению более высокоуровневой информации о сцене.

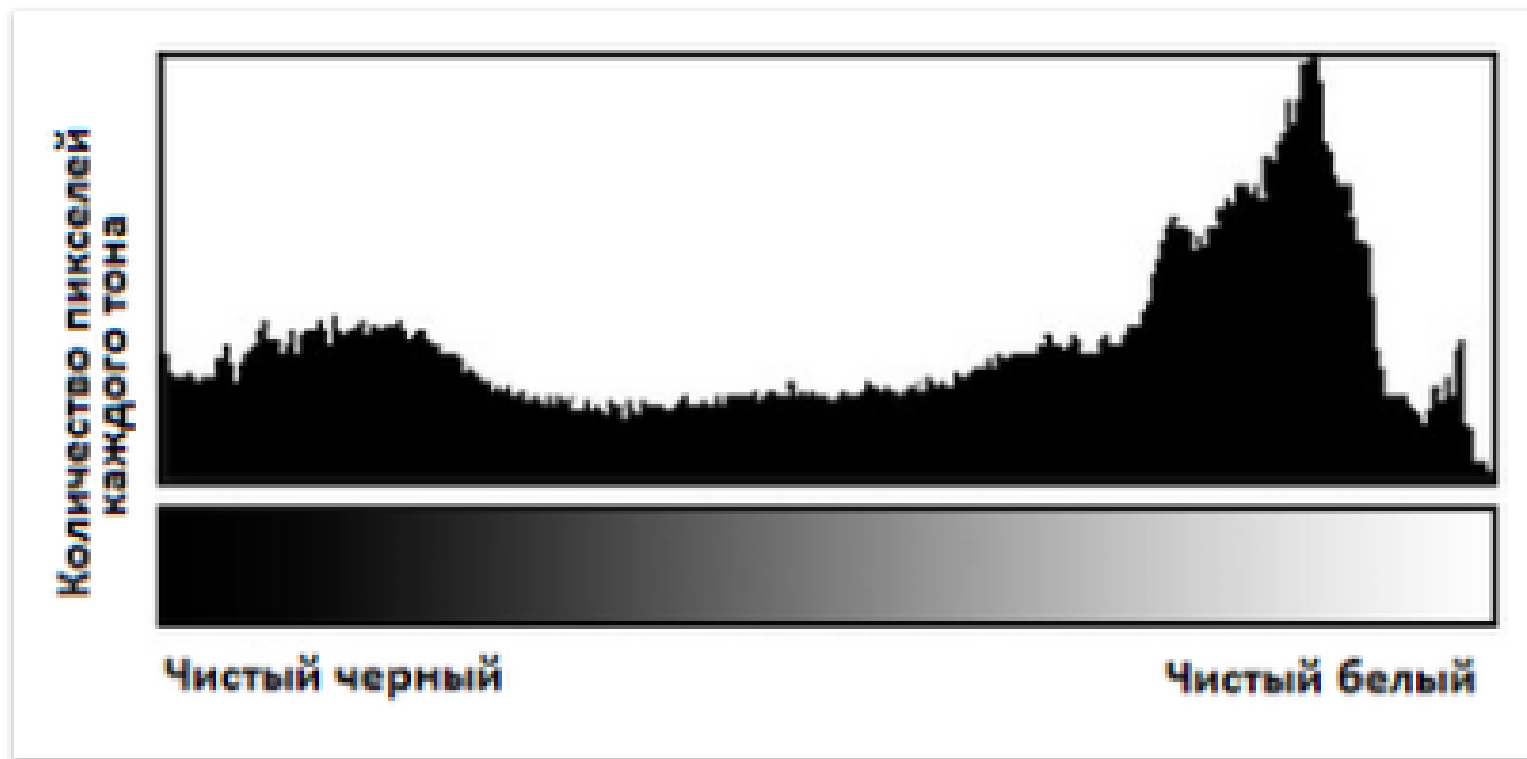
Назначение фильтрации

- Устранение шума в изображениях
- Улучшение качества изображения
- Усиление полезной и подавления нежелательной информации

Изменение контраста изображения

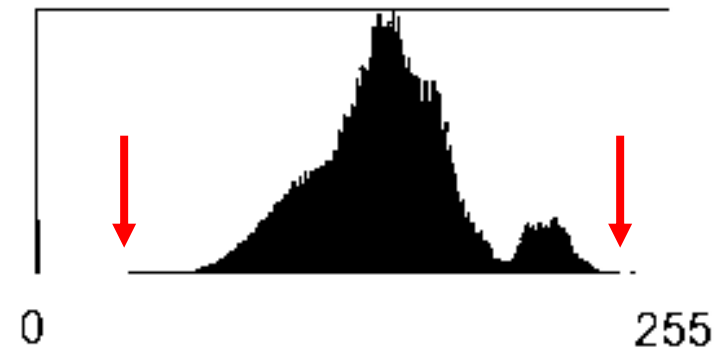
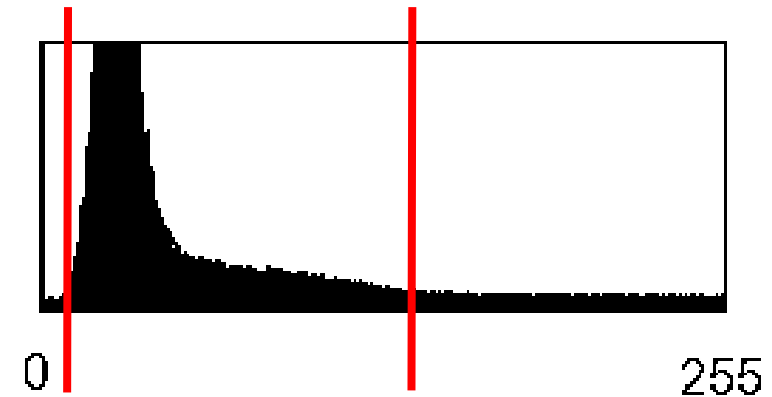
Что такое гистограмма изображения?

Гистограмма – это график распределения яркостей на изображении. На горизонтальной оси - шкала яркостей тонов от черного до белого, на вертикальной оси - число пикселей заданной яркости.



Что может не устраивать в изображении:

- Узкий или смещенный диапазон яркостей пикселей (тусклое или «пересвеченное» изображение)
- Концентрация яркостей вокруг определенных значений, неравномерное заполнение диапазона Яркостей (узкий диапазон - тусклое изображение)



При коррекции к изображению применяется преобразование яркостей, компенсирующий нежелательный эффект:

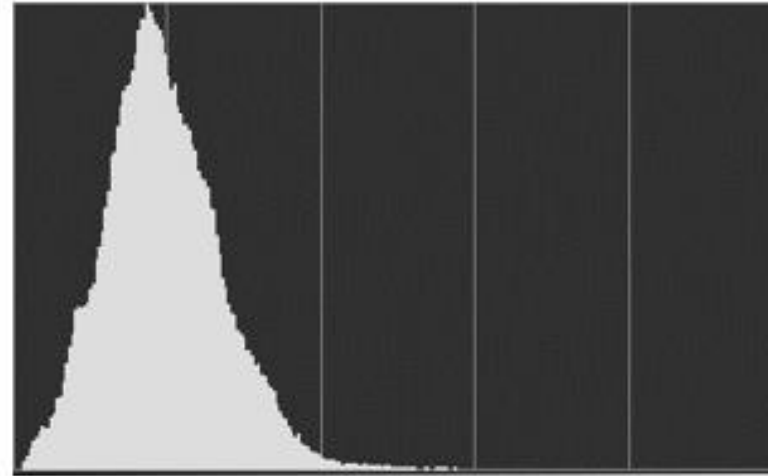
$$x = f^{-1}(y)$$

y – яркость пикселя на исходном изображении,
 x – яркость пикселя после коррекции.

Затемненное изображение (а) и соответствующая ему гистограмма (б);
засвеченное изображение (в) и соответствующая ему гистограмма (г).



а



б



в



г

Изменение контраста изображения

Существует три основных метода повышения контраста изображения:

1. **Линейная** растяжка гистограммы (линейное контрастирование),
2. **Нормализация** гистограммы,
3. **Выравнивание** (линеаризация или эквализация, equalization) гистограммы.

1. Линейная коррекция

Компенсация узкого диапазона яркостей – линейное растяжение:

$$f^{-1}(y) = (y - y_{\min}) * \frac{(255 - 0)}{(y_{\max} - y_{\min})}$$

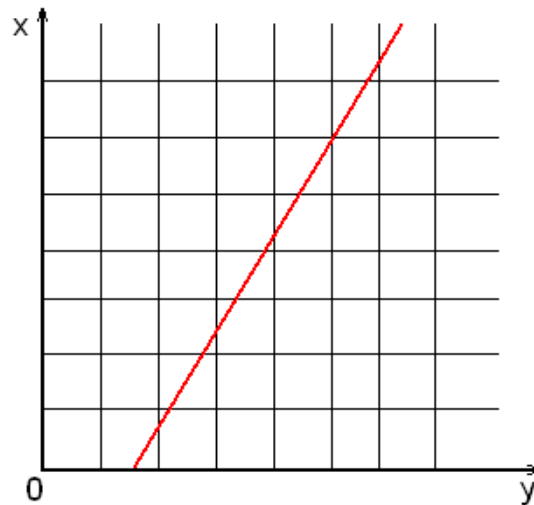
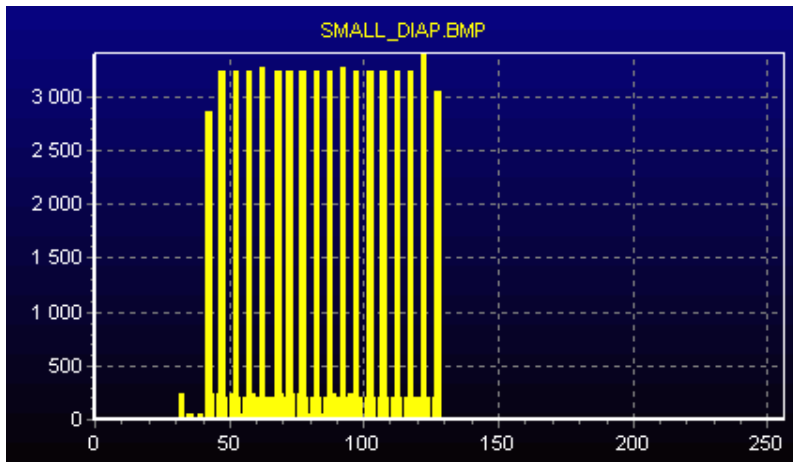
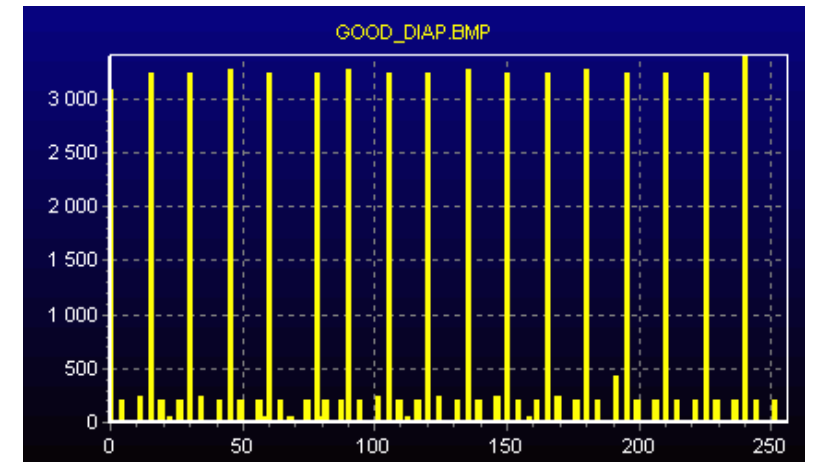
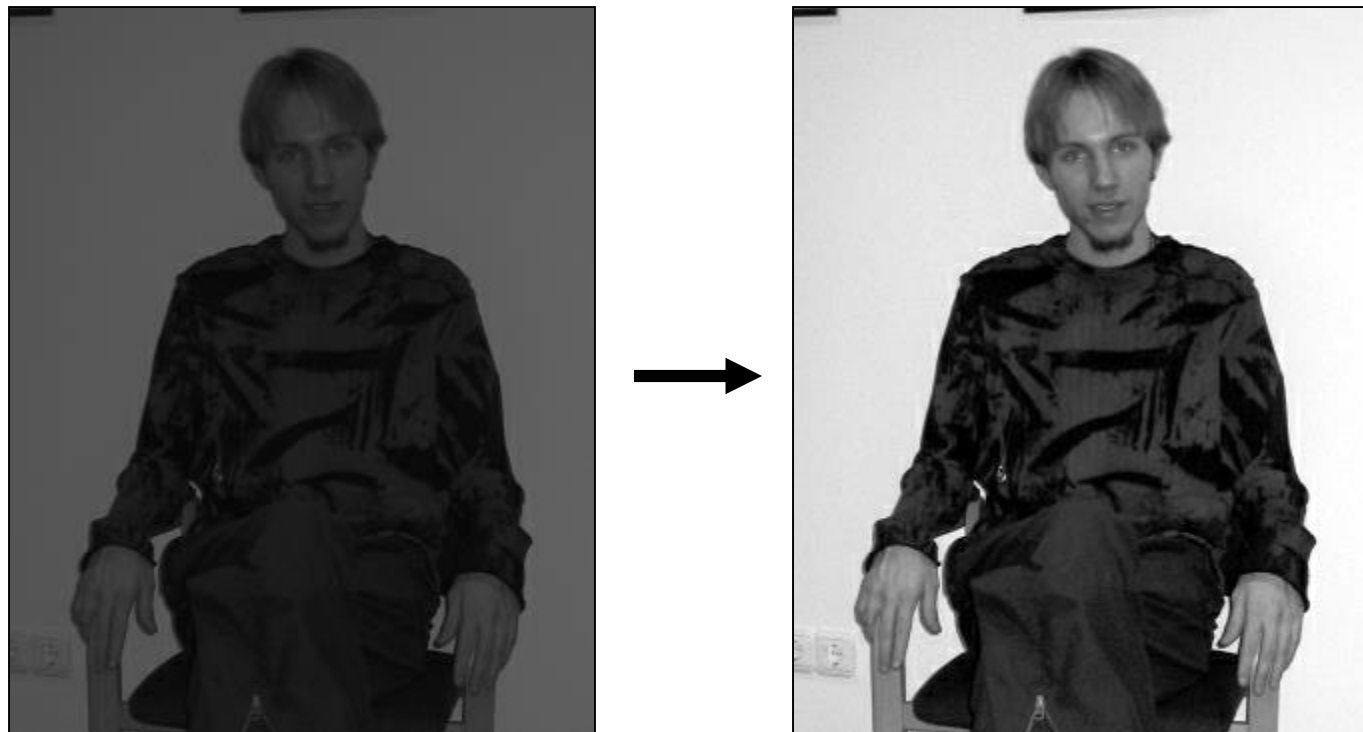


График функции $f^{-1}(y)$



Линейная коррекция

Линейное растяжение – «как **AutoContrast** в Photoshop»



НО линейная коррекция помогает не всегда!



Почему?

Нелинейная коррекция

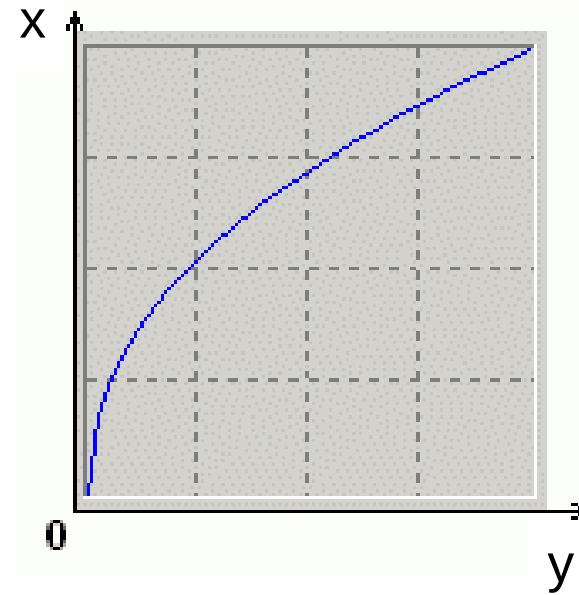


График функции $f^{-1}(y)$

Нелинейная коррекция

Нелинейная компенсация недостаточной контрастности

Часто применяемые функции:

- **Гамма-коррекция**

Изначальная цель – коррекция для правильного отображения на мониторе.

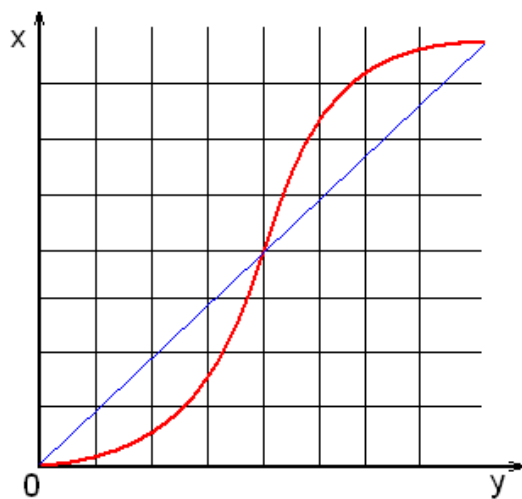
$$y = c \cdot x^\gamma$$

- **Логарифмическая**

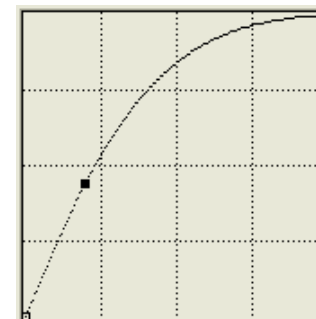
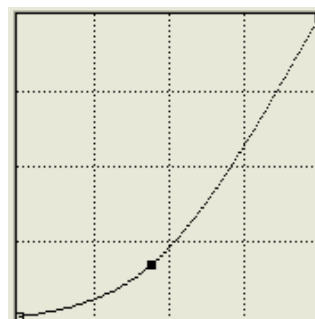
Цель – сжатие динамического диапазона при визуализации данных

$$y = c \cdot \log(1 + x)$$

Гамма-коррекция



Инструмент «Кривые» в Photoshop

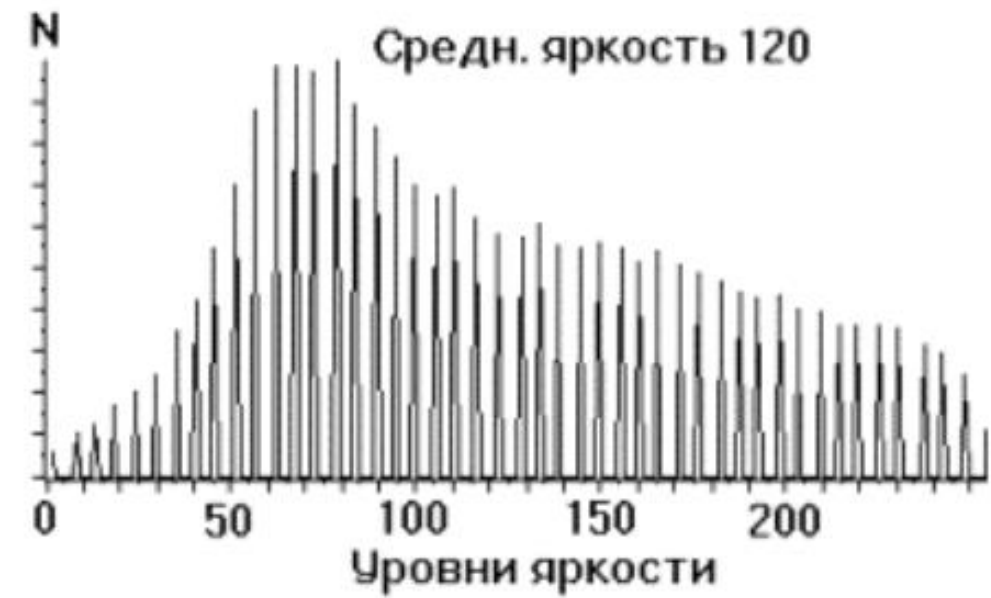
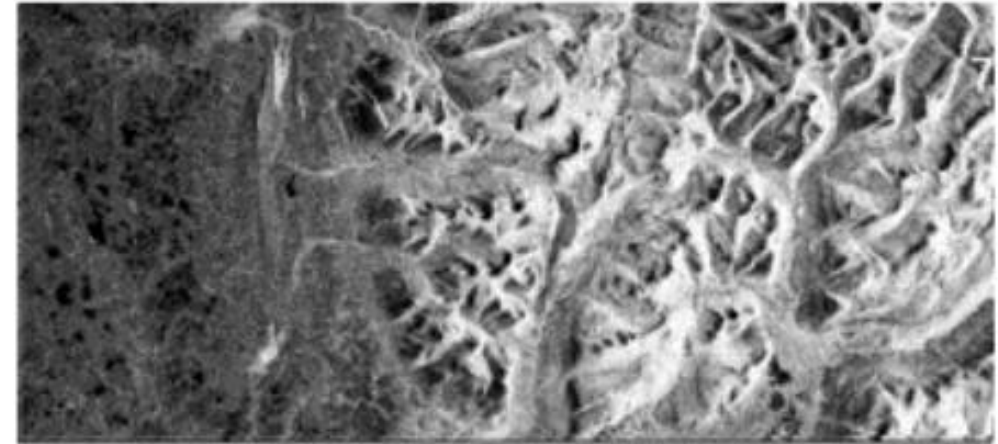
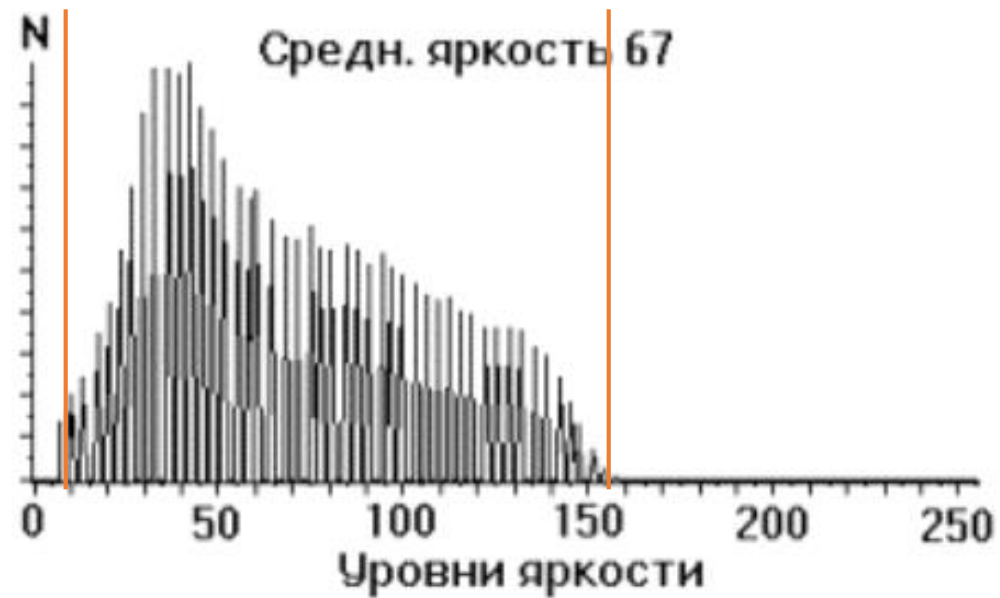
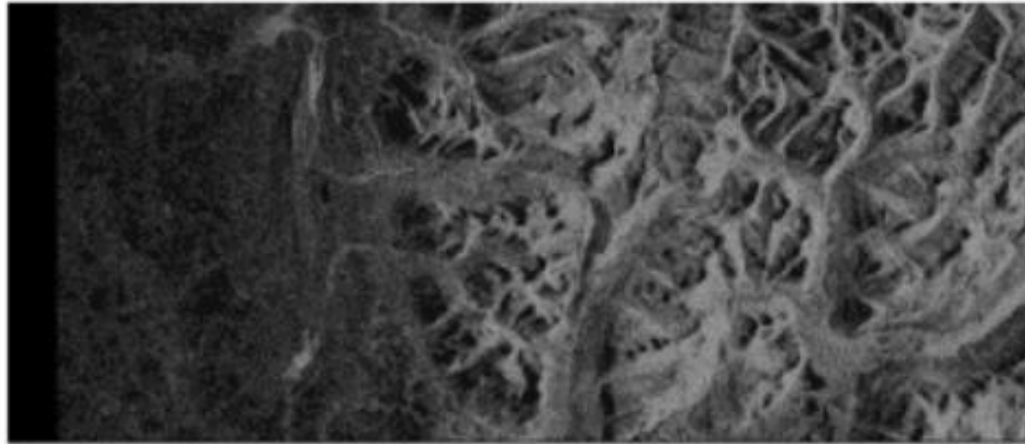


Графики функции $f^{-1}(y)$

2. Нормализация гистограммы

Нормализация гистограммы в отличие от линейной коррекции обеспечивает растяжку **не всего диапазона изменения интенсивностей, а только его наиболее информативной части**. Под информативной частью понимается набор пиков гистограммы, т.е. интенсивности, которые чаще остальных встречаются на изображении. Бины, соответствующие редко встречающимся интенсивностям, в процессе нормализации отбрасываются, далее выполняется обычная линейная растяжка получившейся гистограммы.

2. Нормализация гистограммы



3. Выравнивание гистограммы (эквалазия)

Цель выравнивания состоит в том, чтобы все уровни яркости имели бы одинаковую частоту, а гистограмма соответствовала равномерному закону распределения.

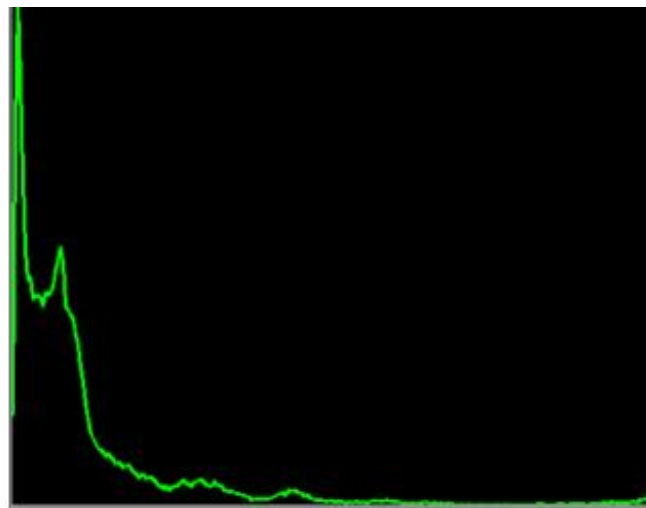
```
void equalizeHist(const Mat& src, Mat& dst)
```



Пример выравнивание гистограммы



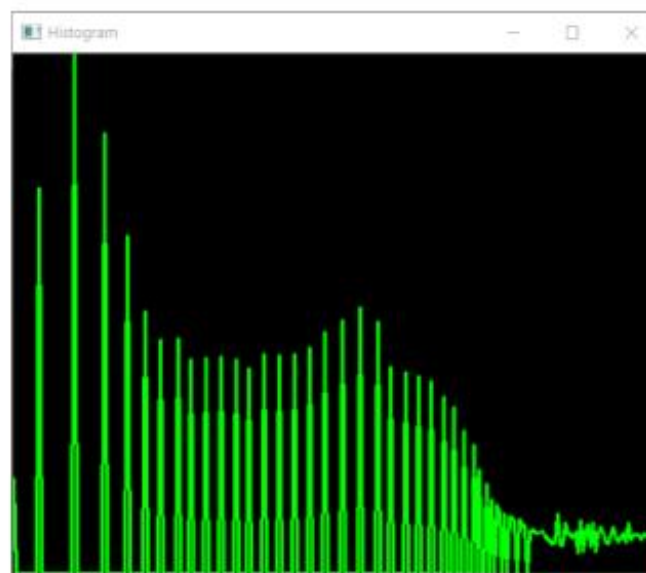
a



б



в



г

Цветовая коррекция

Изменение цветового баланса для компенсации:

- Неверного цветовосприятия камеры
- Цветного освещения



Цветовая коррекция изображений

Растяжение контрастности (“autolevels”)

Идея – растянуть интенсивности по каждому из каналов на весь диапазон;

Реализация:

- Найти минимум, максимум по каждому из каналов:

$$R_{\min}, R_{\max}, G_{\min}, G_{\max}, B_{\min}, B_{\max}$$

- Преобразовать интенсивности:

$$(R - R_{\min}) * \frac{(255 - 0)}{(R_{\max} - R_{\min})}; \quad (G - G_{\min}) * \frac{(255 - 0)}{(G_{\max} - G_{\min})};$$

$$(B - B_{\min}) * \frac{(255 - 0)}{(B_{\max} - B_{\min})};$$

Растяжение контрастности



Исходное
изображение



Эквализация RGB



Эквализация
интенсивности



Применение эквализации к трем каналам **RGB** как правило приводит к не лучшим результатам. Цвета получаются холодными и неестественными. Эквализация по интенсивности в модели **HSI** приводит к лучшим результатам.

HSI или HSL (от англ. hue, saturation, lightness (intensity)) — цветовая модель, в которой цветовыми координатами являются тон, насыщенность и светлота(интенсивность).

Шумоподавление

Причины возникновения шума:

- Несовершенство измерительных приборов
- Хранение и передача изображений с потерей данных



Шум фотоаппарата



Сильное сжатие JPEG

Определение свертки

Будем считать, что задано исходное полутоновое изображение I , и обозначим интенсивности его пикселей $I(x, y)$. Линейный фильтр определяется вещественнозначной функцией F , заданной на растре. Данная функция называется ядром фильтра, а сама фильтрация производится при помощи операции дискретной свертки (взвешенного суммирования) по следующей формуле:

$$I'(x, y) = \sum_i \sum_j F(i, j) \cdot I(x + i, y + j)$$

Свёртка — это операция **вычисления нового значения** заданного пикселя, при которой **учитываются значения окружающих его соседних пикселей**.

Главным элементом свёртки является т.н. **ядро свёртки** — это матрица (произвольного размера и отношения сторон; чаще всего используется квадратная матрица (по-умолчанию, 3x3)).

Определение свертки

Ядро свёртки — это числовая матрица с подобранными коэффициентами. Эти коэффициенты называют степенями влияния или “ценностями”, определяющими вес пикселей из рассматриваемой окрестности.

Входное изображение

12	14	41
43	84	24
2	12	12

Ядро фильтра

0,5	0,75	0,5
0,75	1,0	0,75
0,5	0,75	0,5

$\times$

$=$

Результат

$$= \left[\begin{array}{l} 12 \cdot 0,5 + 14 \cdot 0,75 + 41 \cdot 0,5 + \\ + 43 \cdot 0,75 + 84 \cdot 1,0 + 24 \cdot 0,75 + \\ + 2 \cdot 0,5 + 12 \cdot 0,75 + 12 \cdot 0,5 \end{array} \right] \times \frac{1}{\text{div}} = 31,2$$

Коэффициент нормирования $\text{div} = 6$

Источник [Матричные фильтры обработки изображений / Хабр \(habr.com\)](https://habr.com/ru/articles/444444/)

Хорошая визуализация процессов свертки [Визуальное объяснение ядер изображений \(setosa.io\)](https://setosa.io/visual/conv-demo.html)

Для вычисления сверток в библиотеке OpenCV присутствует функция **filter2D()**:

```
void filter2D(const Mat& src, Mat& dst, int ddepth,  
             const Mat& kernel, Point anchor=Point(-1, -1),  
             double delta=0, int borderType=BORDER_DEFAULT)
```

```
const float kernelData[] = { -1.0f, -1.0f, -1.0f,  
                             -1.0f, 9.0f, -1.0f,  
                             -1.0f, -1.0f, -1.0f };  
const Mat kernel(3, 3, CV_32FC1, (float*)kernelData);  
filter2D(src, dst, -1, kernel);
```

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Какое изображение
получим в итоге
применения фильтра с
таким ядром

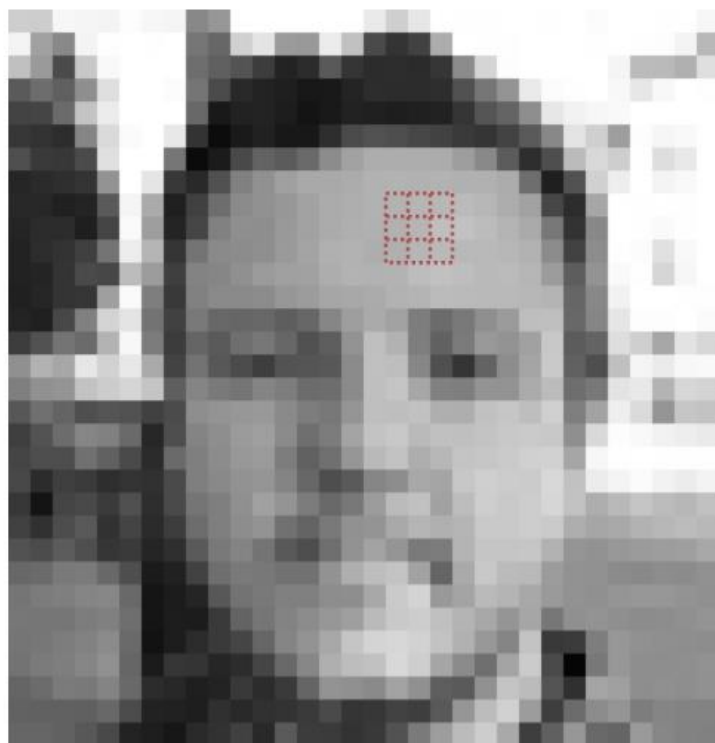
?

0	-1	0
-1	5	-1
0	-1	0

Какое изображение
получим в итоге
применения фильтра с
таким ядром

?

повышения контраста



input image

$$\begin{pmatrix}
 179 \times 0 + 182 \times -1 + 184 \times 0 \\
 + 178 \times -1 + 182 \times 5 + 191 \times -1 \\
 + 177 \times 0 + 183 \times -1 + 186 \times 0
 \end{pmatrix}
 = 176$$

kernel:

sharpen



output image

Пример фильтра улучшения чёткости

$$\text{kernel} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



-2	-1	0
-1	1	1
0	1	2

Ядро тиснения создает иллюзию глубины, подчеркивая различия пикселей в заданном направлении. В данном случае в направлении вдоль линии от левого верхнего угла к правому нижнему.



Фильтры. Сглаживание изображений

Сглаживание или размытие – это одна из самых простых и часто используемых операций обработки изображений. Как правило, размытие применяется, чтобы уменьшить шумы или артефакты.

В библиотеке OpenCV реализовано несколько функций размытия изображения, к примеру:

- `blur()`,
- `GaussianBlur()`
- `medianBlur()`

```
void blur(const Mat& src, Mat& dst, Size ksize,  
          Point anchor=Point(-1, -1),  
          int borderType=BORDER_DEFAULT)
```

```
void GaussianBlur(const Mat& src, Mat& dst, Size ksize,  
                  double sigmaX, double sigmaY=0,  
                  int borderType=BORDER_DEFAULT)
```

```
void medianBlur(const Mat& src, Mat& dst, int ksize)
```

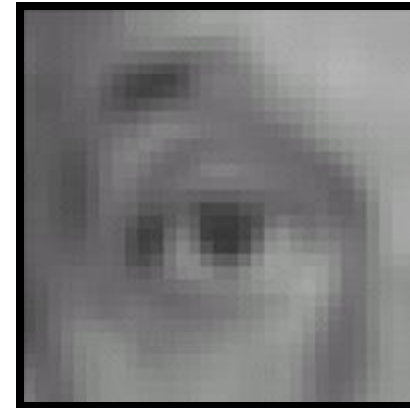
Простейшие фильтры. Blur



Original

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1

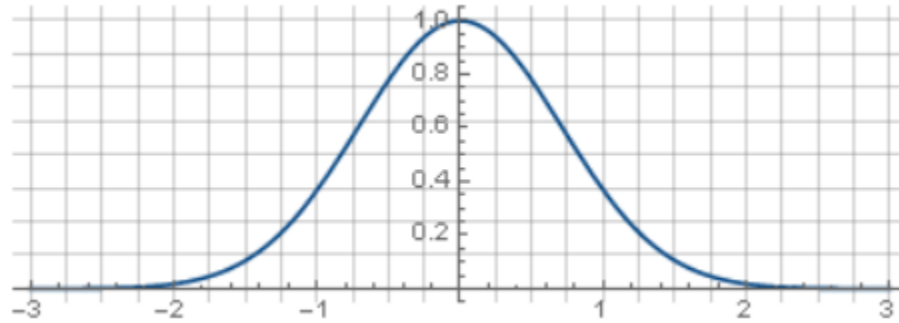


Blur (with a
box filter)

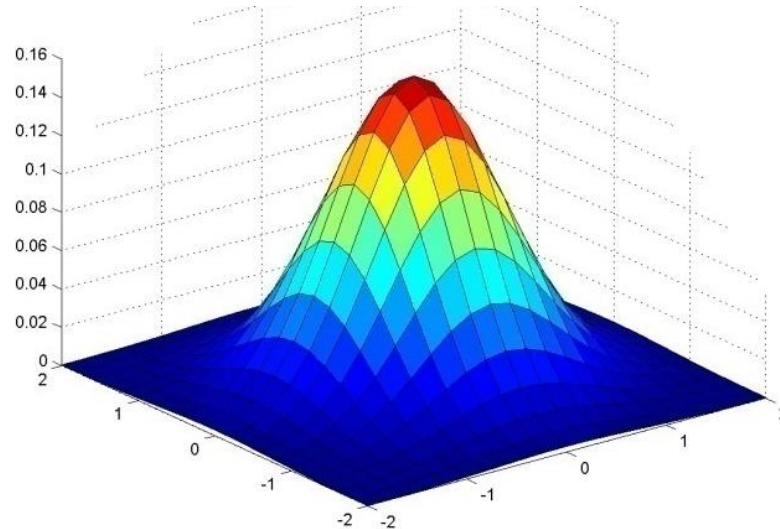
$$K = \frac{1}{\text{ksize.width} * \text{ksize.height}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \\ \dots & & & & & \\ 1 & 1 & 1 & \dots & 1 & 1 \end{bmatrix}$$

Фильтр Гаусса (Gaussian Blur)

Фильтр называется Gaussian, потому что он строится из функции, известной как гауссиана,



$$e^{-x^2}$$

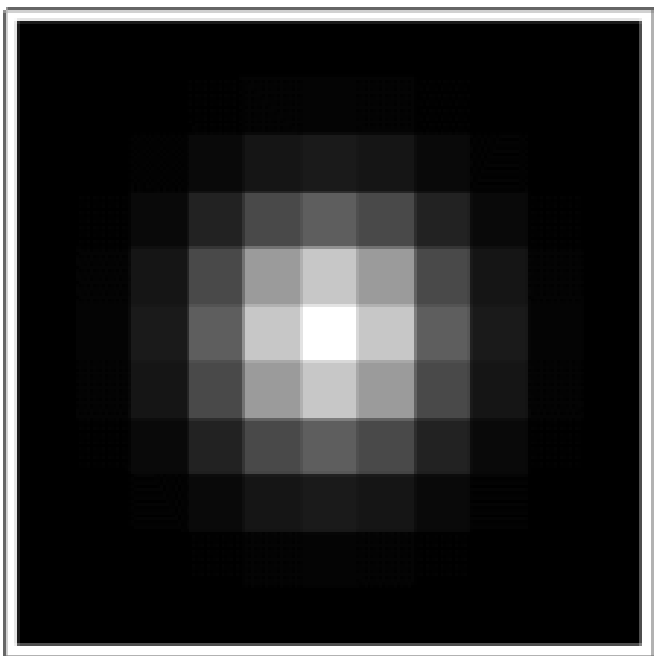


$$e^{-(x^2+y^2)}$$

Фильтр Гаусса (Gaussian Blur)

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Для нормализации по координатам
введена **сигма**
(среднеквадратическое
отклонение), которая определяет
степень сжатия гауссианы



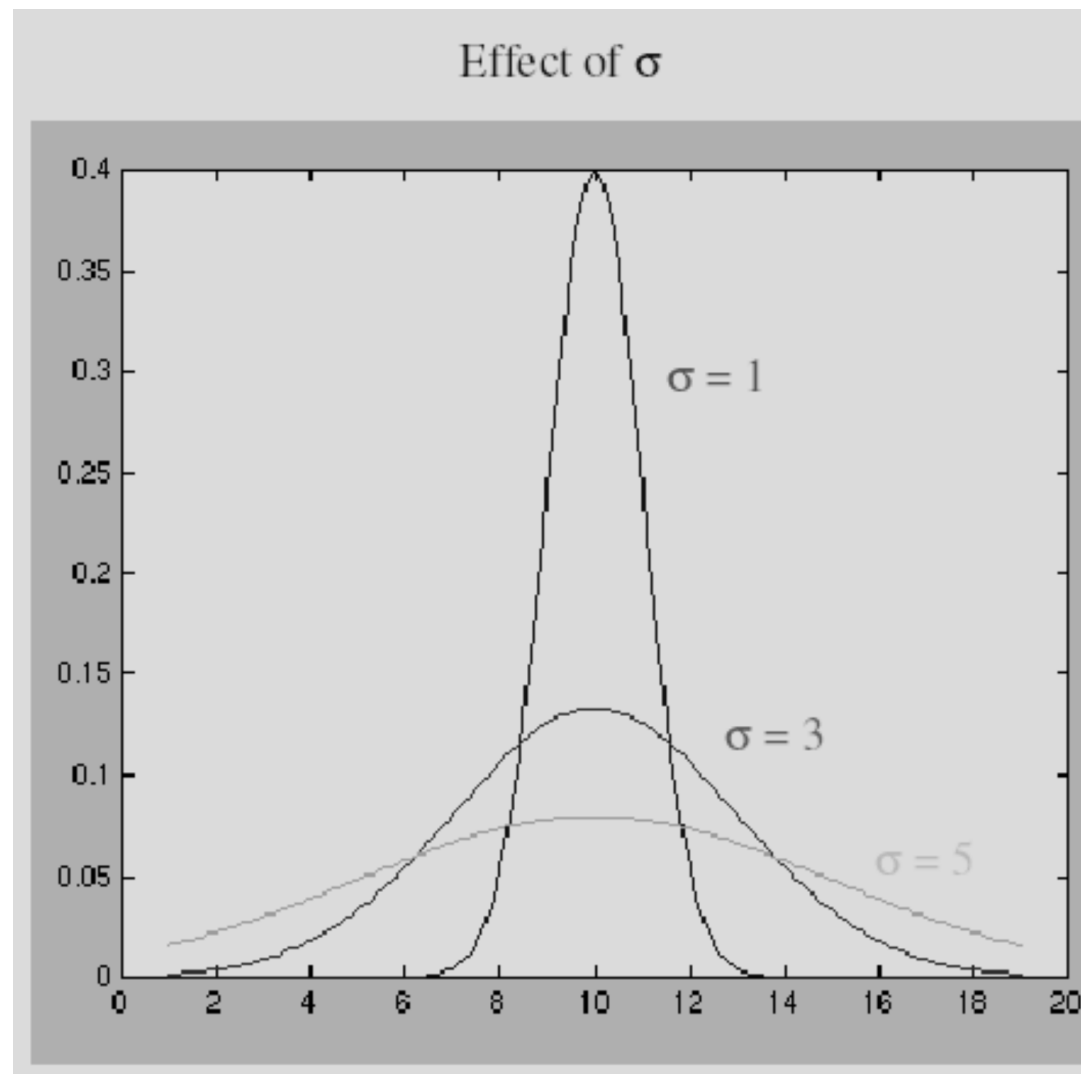
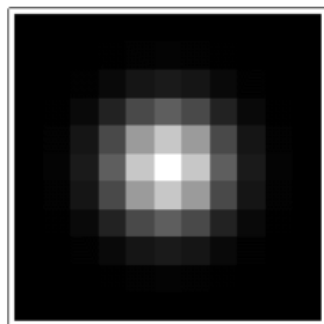
Ядро фильтра Гаусса

0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

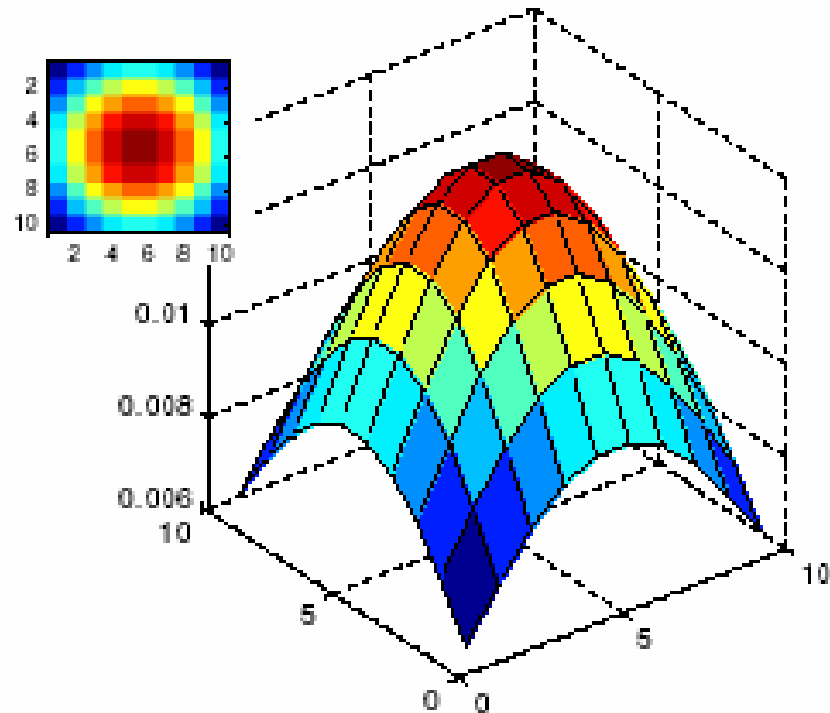
5 x 5, $\sigma = 1$

```
void GaussianBlur(const Mat& src, Mat& dst, Size ksize,  
                  double sigmaX, double sigmaY=0,  
                  int borderType=BORDER_DEFAULT)
```

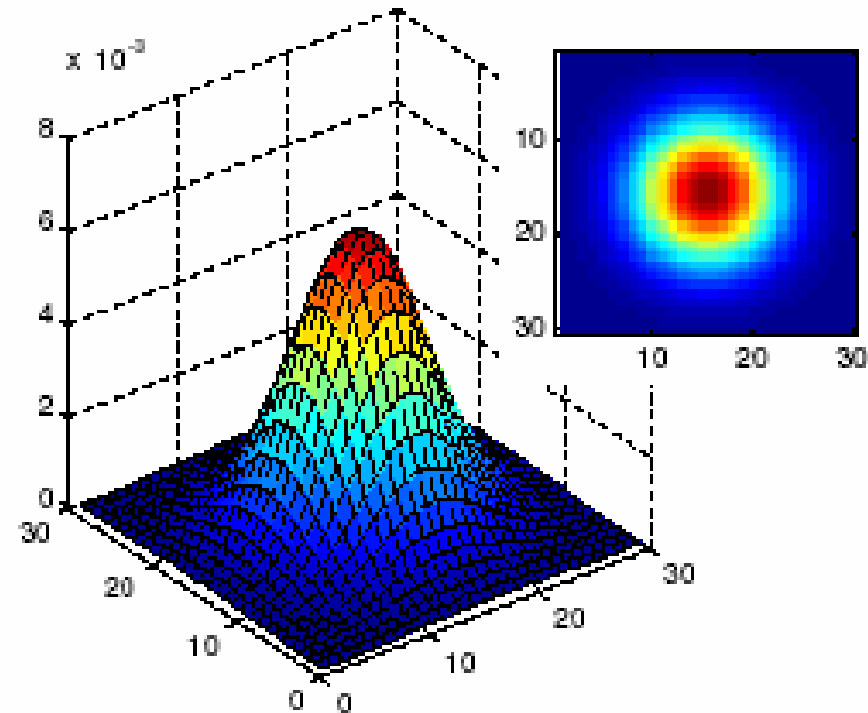
зависимость размера
сглаживающей области от
сигма



Выбор размера ядра



$\sigma = 5$ kernel 10x10



$\sigma = 5$ kernel 30x10



Original



Salt and pepper noise



Impulse noise

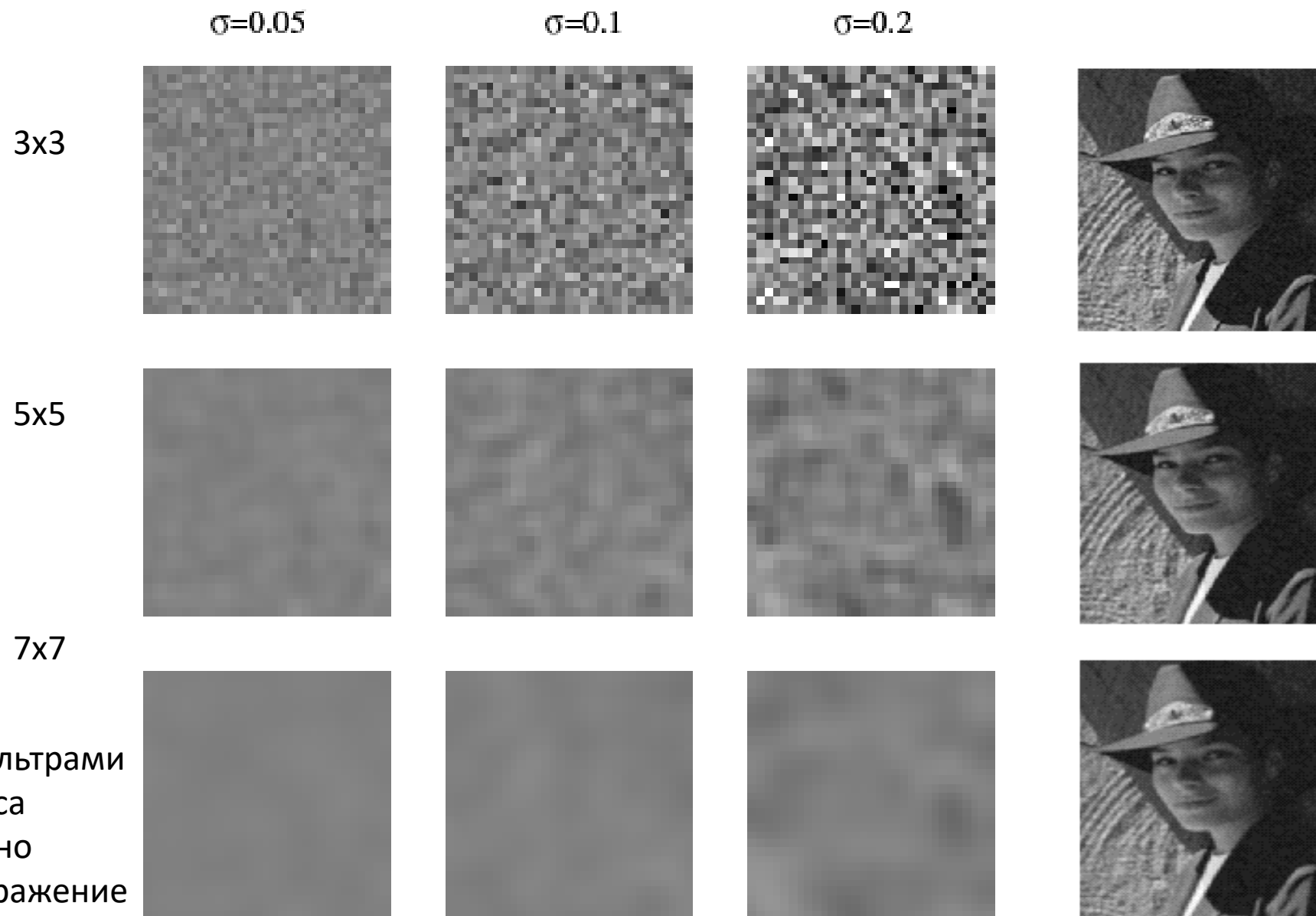


Gaussian noise

Виды шума

- **Соль и перец:**
случайные черные и белые пиксели
- **Импульсный:**
случайные белые пиксели
- **Гауссов:** колебания яркости, распределенные по нормальному закону

Подавление гауссова шума



Сглаживание фильтрами
большого радиуса
подавляет шум, но
размывает изображение

Подавление шума «соль и перец»

3x3



5x5



7x7

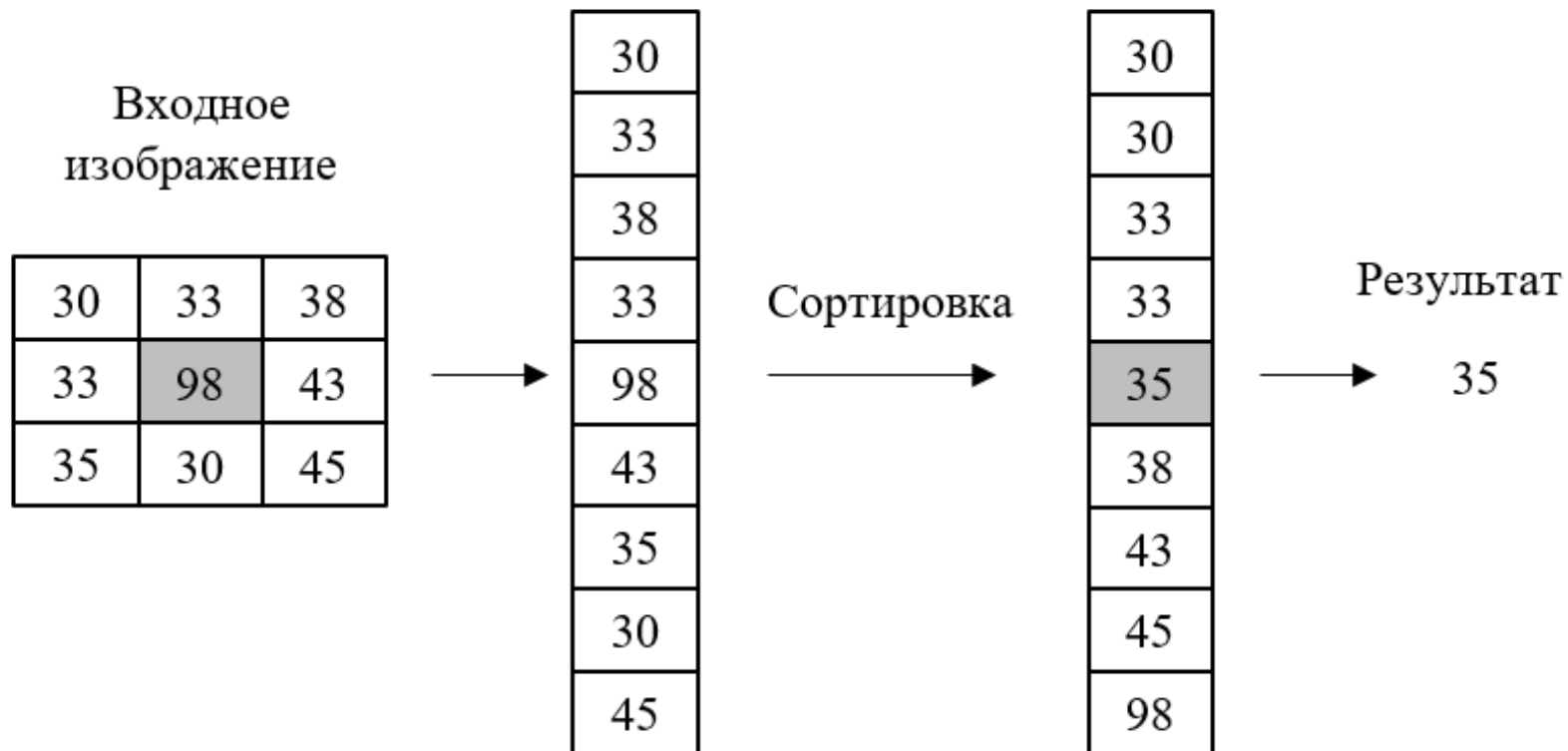


Чем плох результат?

Медианный фильтр

Фильтр работает с матрицами различного размера, но в отличие от матрицы свёртки, размер матрицы влияет только на количество рассматриваемых пикселей.

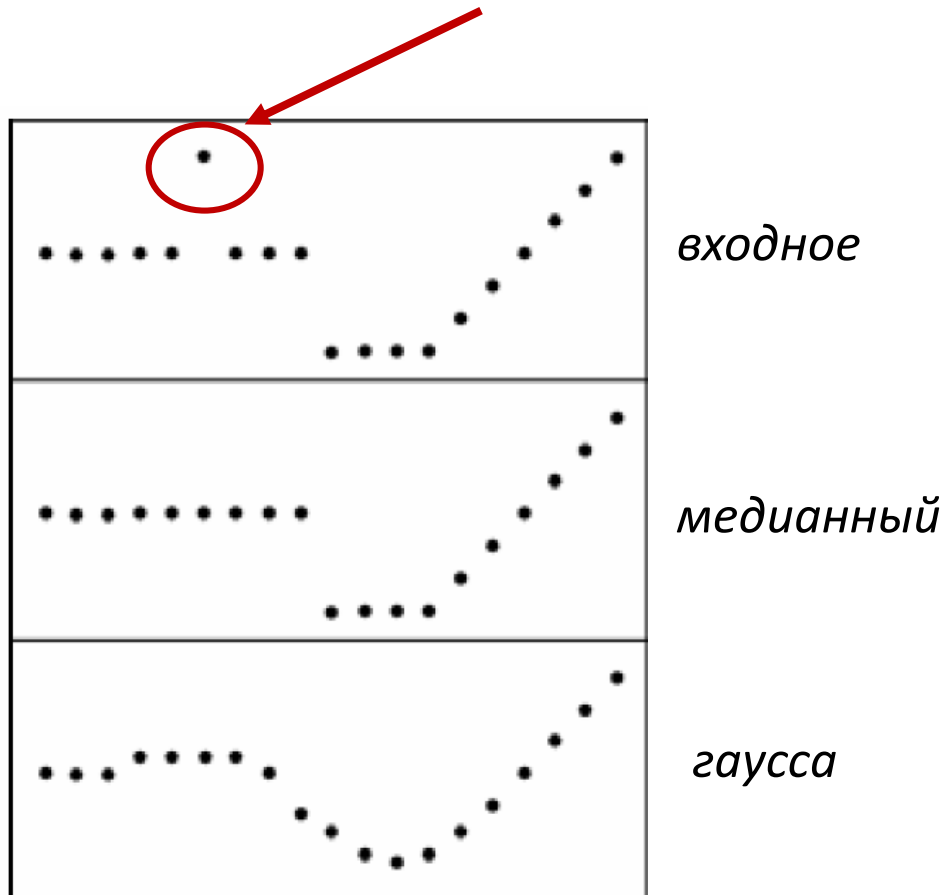
Пиксели, которые «попадают» в матрицу вокруг текущего пикселя, сортируются, и выбирается срединное значение из отсортированного массива. По сути, определяется медиана в отсортированном наборе данных. Это значение и является выходным для текущего пикселя.



Медианный фильтр

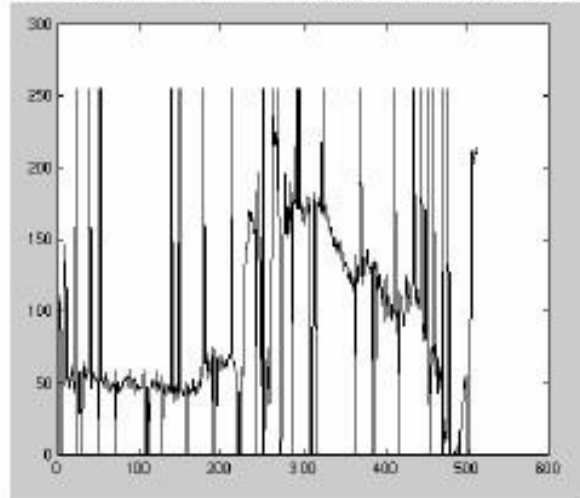
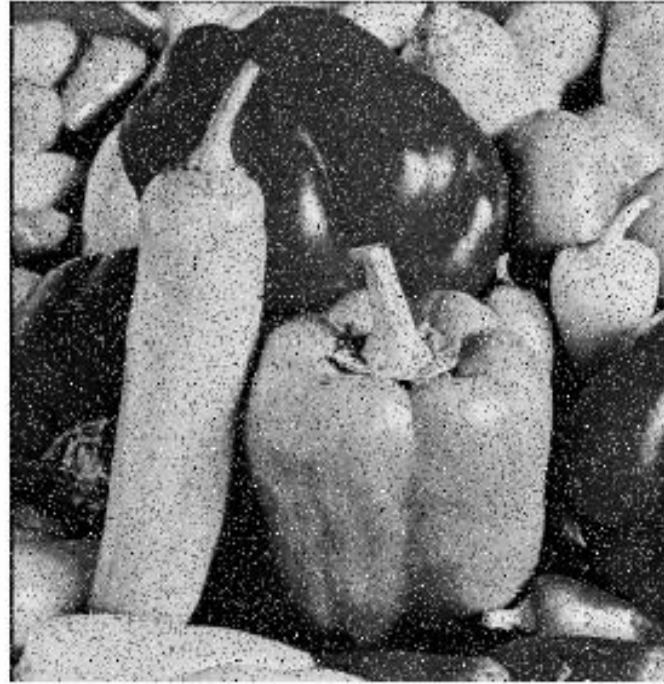
В чем преимущество медианного фильтра перед фильтром гаусса?

Устойчивость к выбросам (outliers)

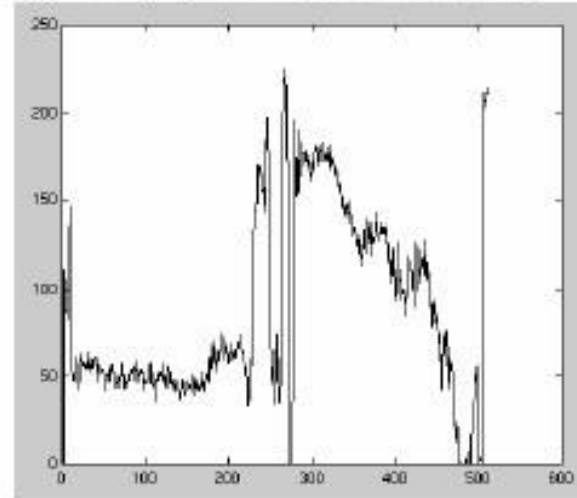


Преимущество медианной фильтрации перед фильтрами размытия (***blur()***, ***box()***, ***GaussianBlur()***) заключается в том, что «битый» пиксель на темном фоне будет заменен на темный, а не «размазан» по окрестности.

Salt-and-pepper noise

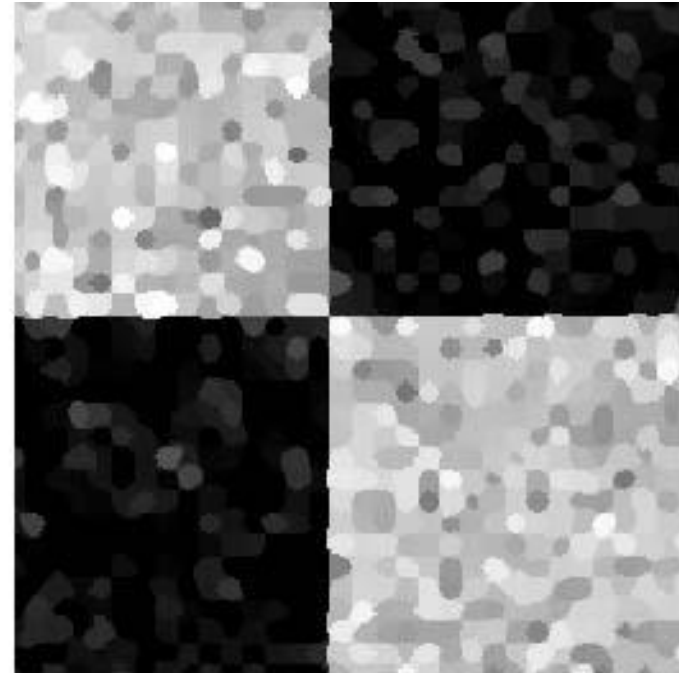
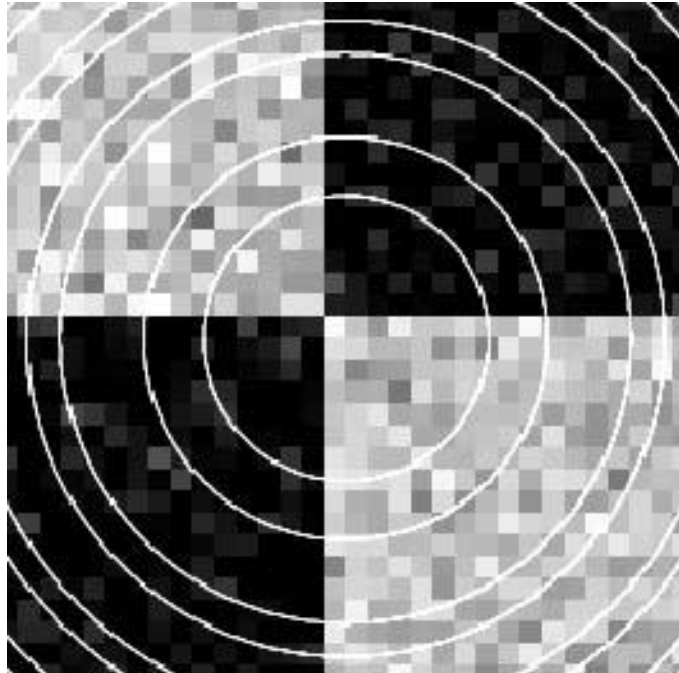


Median filtered



Медианный фильтр

Результат применения медианного фильтра к изображению с артефактами в виде тонких светлых линий.



Сравнение фильтров

3x3

5x5

7x7

Гауссов



Медианный

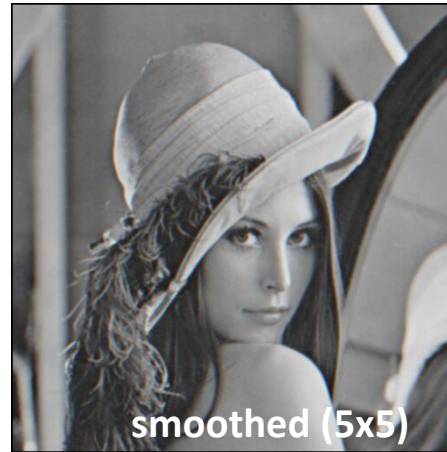


Повышение резкости

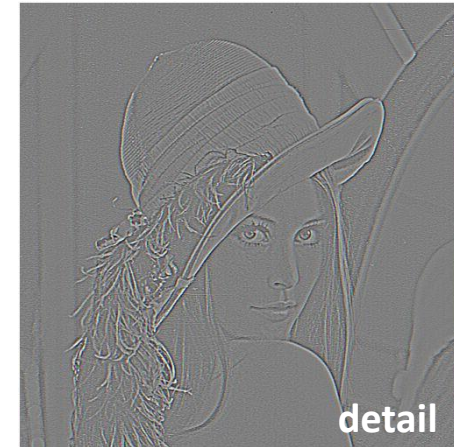
Что теряется при сглаживании?



—



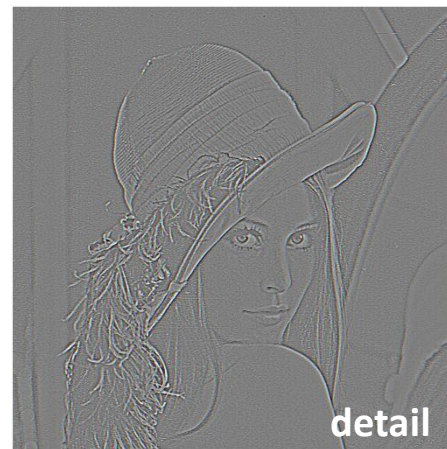
=



Добавим дополнительно высокие частоты:



+ α



=



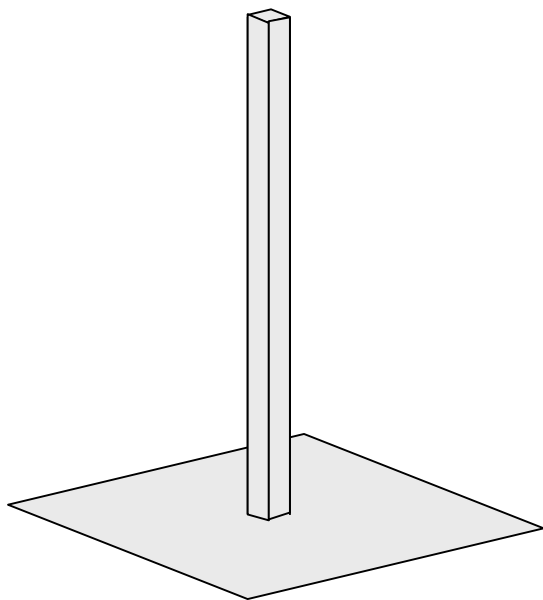
Повышение резкости (Фильтр Unsharp)

$$f + \alpha(f - f * g) = (1 + \alpha)f - \alpha f * g = f * ((1 + \alpha)e - \alpha g)$$

↑
изображение

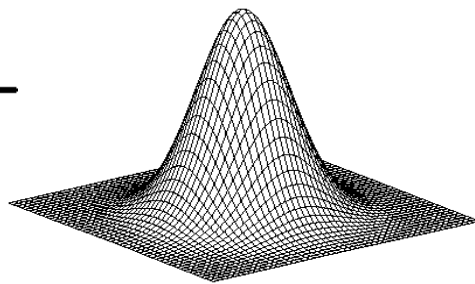
↑
сглаженное
изображение

↑
Единичный
Фильтр



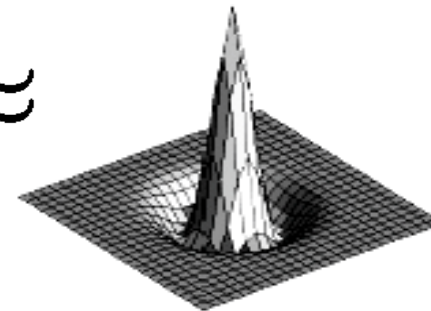
Единичный фильтр

—



Гауссов

≈



Лапласиан гауссиана

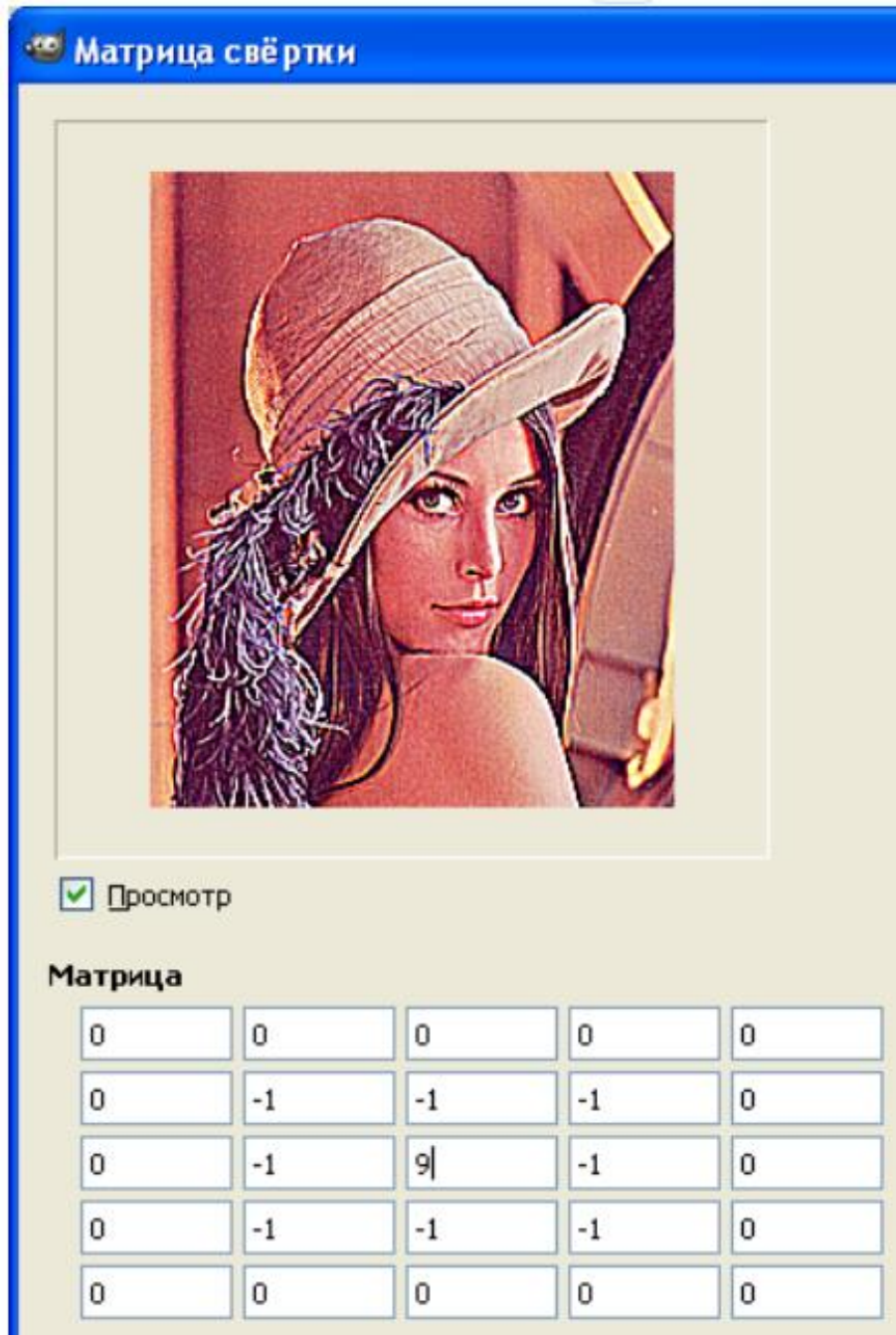
Пример фильтра улучшения чёткости

Ядро свертки

$$\frac{1}{10} \cdot \begin{vmatrix} -1 & -2 & -1 \\ -2 & 22 & -2 \\ -1 & -2 & -1 \end{vmatrix}$$



В некоторых графических редакторах (в частности в программе *GIMP*) есть фильтр «Матрица свёртки», который упрощает поиск необходимого матричного преобразования.



Тиснение



$$\begin{vmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{vmatrix} \text{ либо}$$

$$\begin{vmatrix} 0 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & 0 \end{vmatrix}$$

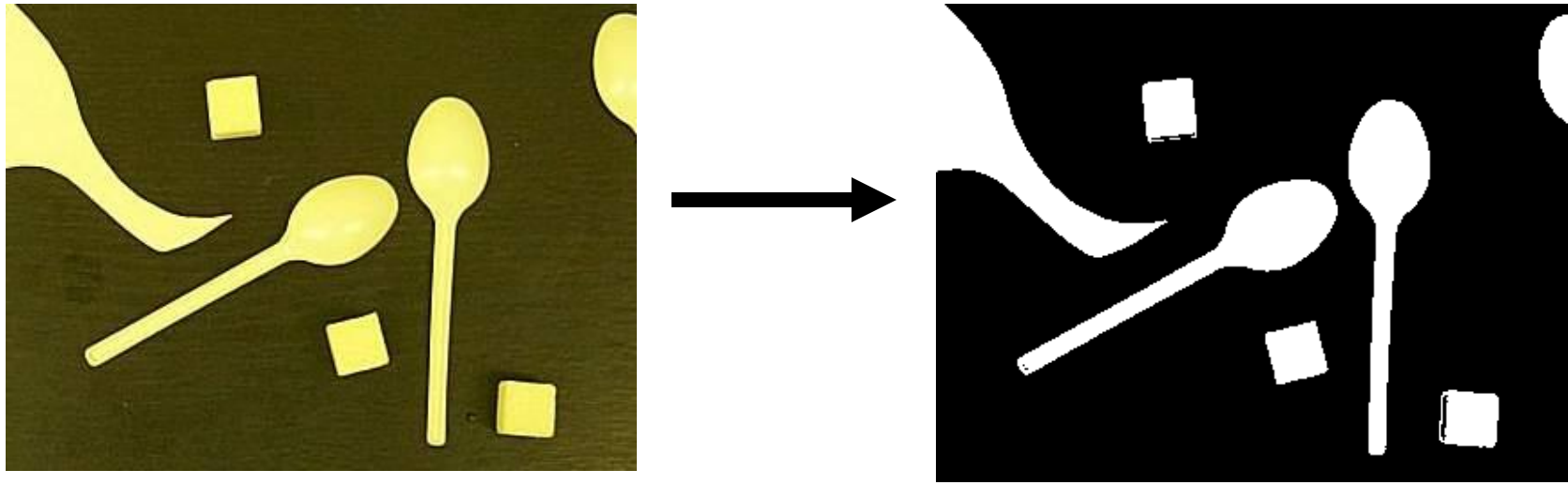
Выделение границ



$$\begin{vmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{vmatrix}$$

Бинаризация изображений

Бинаризация изображений



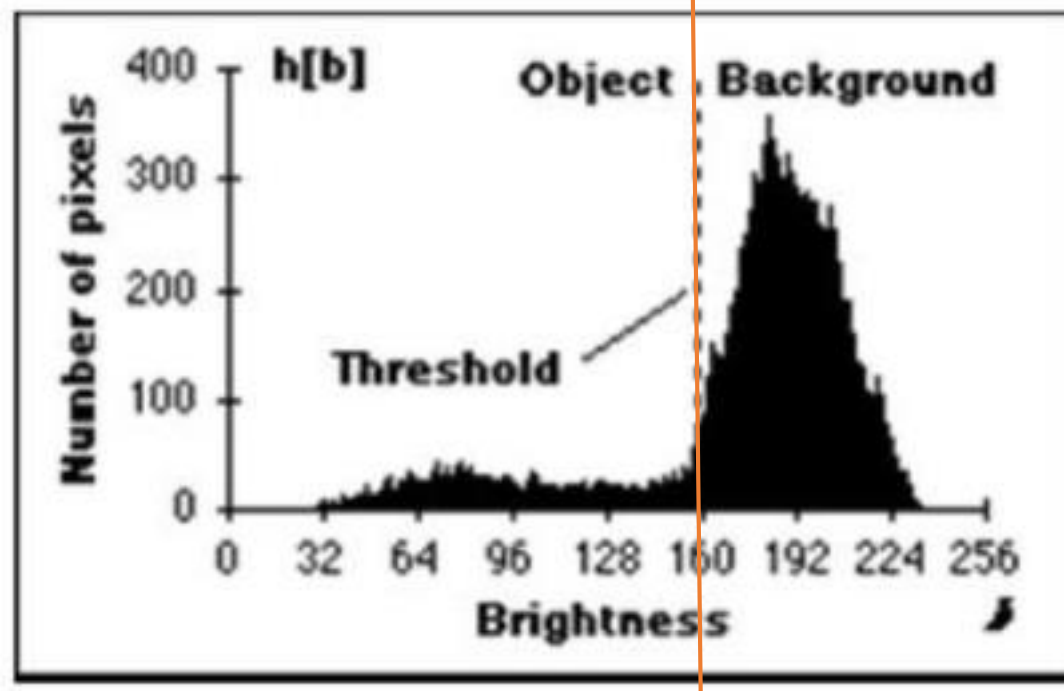
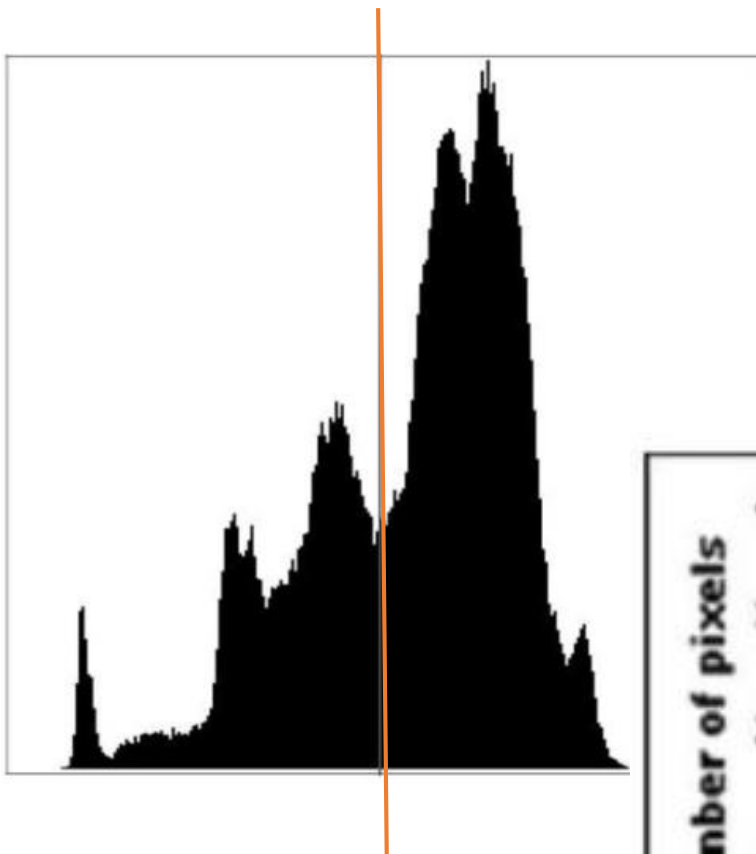
- Бинаризация – построение бинарного изображения по полутоновому / цветному
 - *пороговая фильтрация, которая сводится к выделению областей, яркость которых выше/ниже некоторого порога*
- Пиксель бинарного изображения может значения 0 и 1

Смысл

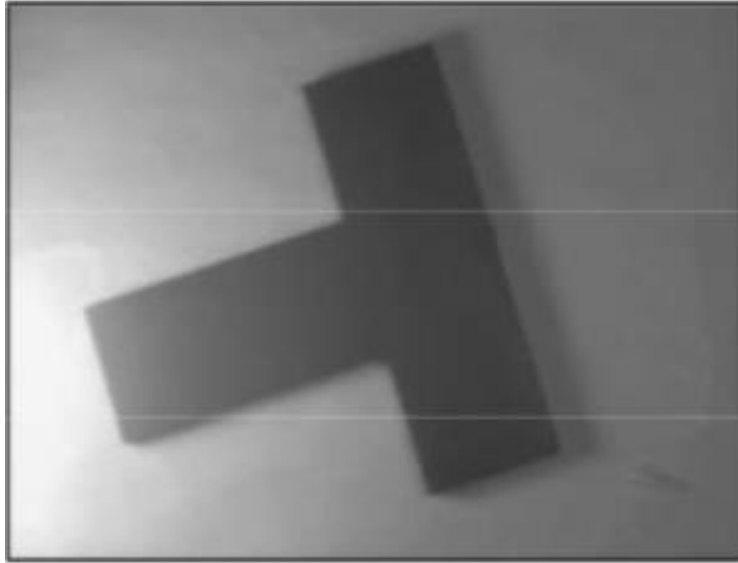
- Разделить изображение на фон и контрастные объекты

Бинаризация изображений

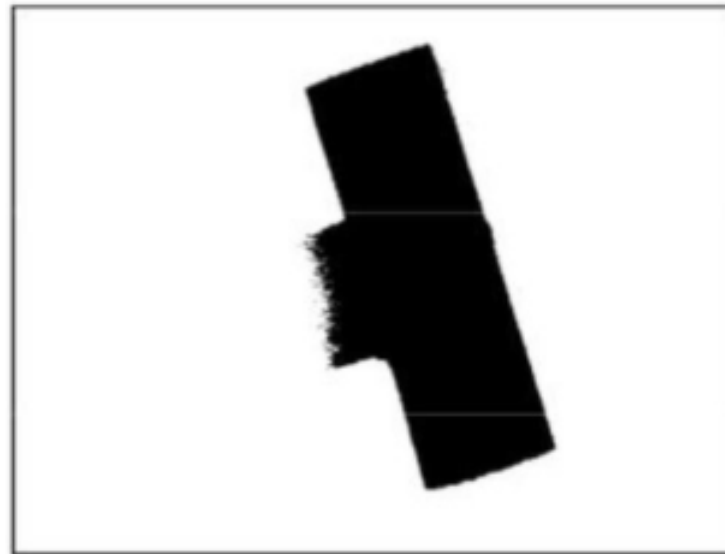
Пороговая
бинаризация



Пороговая бинаризация



В данном случае привела к
неудовлетворительному
результату



Адаптивная бинаризация

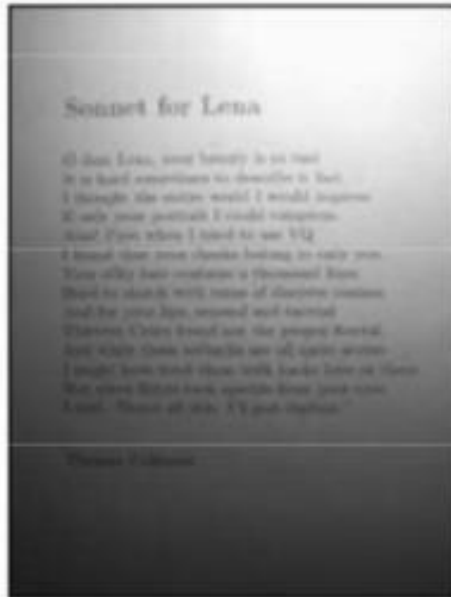
Необходима в случае неравномерной яркости фона/объекта.

- Для каждого пикселя изображения $I(x, y)$:
 1. В окрестности пикселя радиуса r высчитывается индивидуальный для данного пикселя порог T ;
 2. Если $I(x, y) > T + C$, результат 1, иначе 0;

Варианты выбора T :

- $T = mean$
- $T = median$
- $T = (min + max) / 2$

Адаптивная бинаризация



Исходное



$r=7, C=0$



$r=7, C=7$



$r=75, C=10$

r – радиус окрестности пикселя,
 C – производная константа

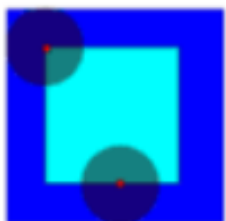
Если $I(x, y) > T + C$, результат 1, иначе 0

Операции математической морфологии

Широко известный способ - устранение шума с помощью операций математической морфологии:

- Сужение (**erosion**) *увеличиваются тёмные области*
- Расширение (**dilation**) *увеличиваются светлые области*
- Закрывание (closing)
- Раскрытие (opening)

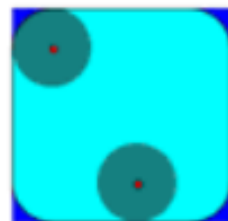
Erosion



Dilation



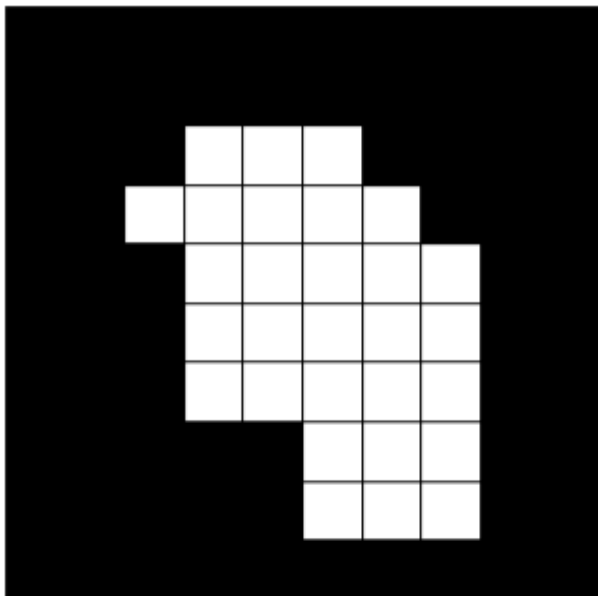
Opening



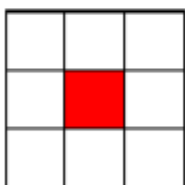
Closing



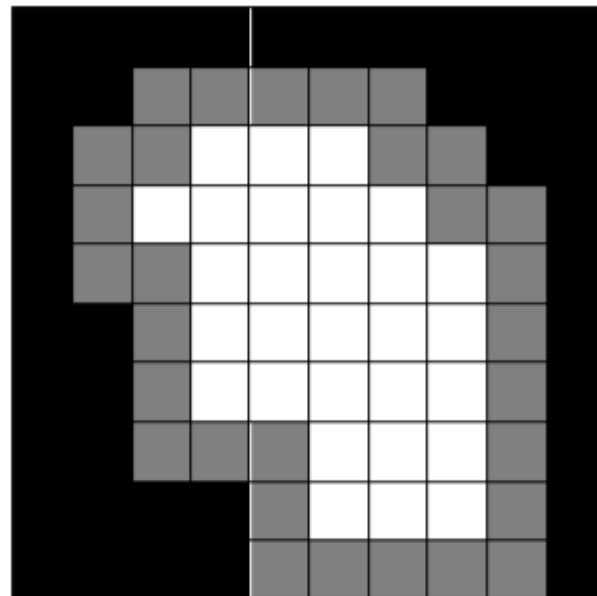
*Математическая морфология
– это теория и техника
анализа и обработки
геометрических структур*



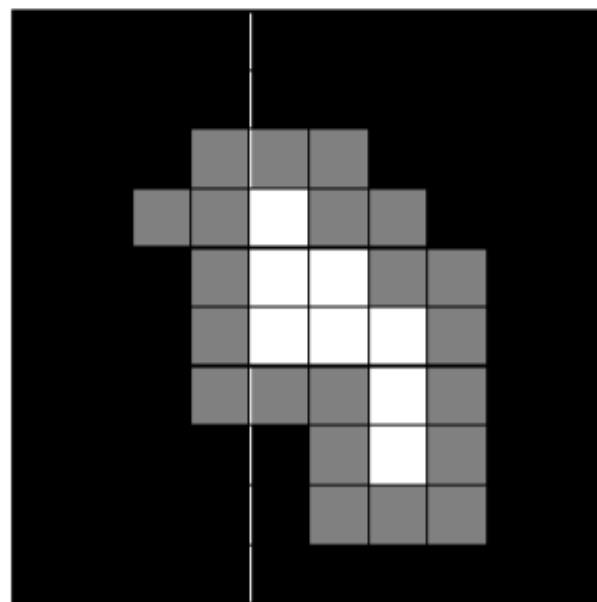
a) исходное изображение



b) шаблон (центр – ведущий элемент)



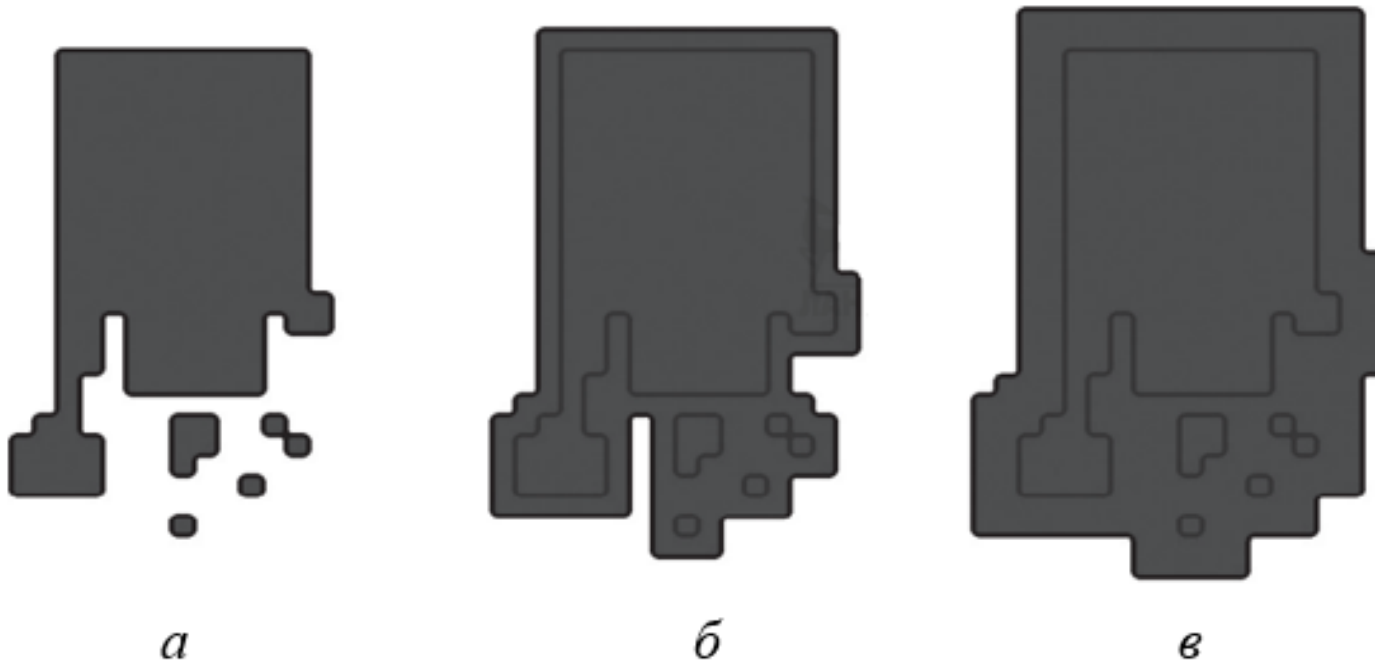
c) результат дилатации



d) результат эрозии

Дилатация

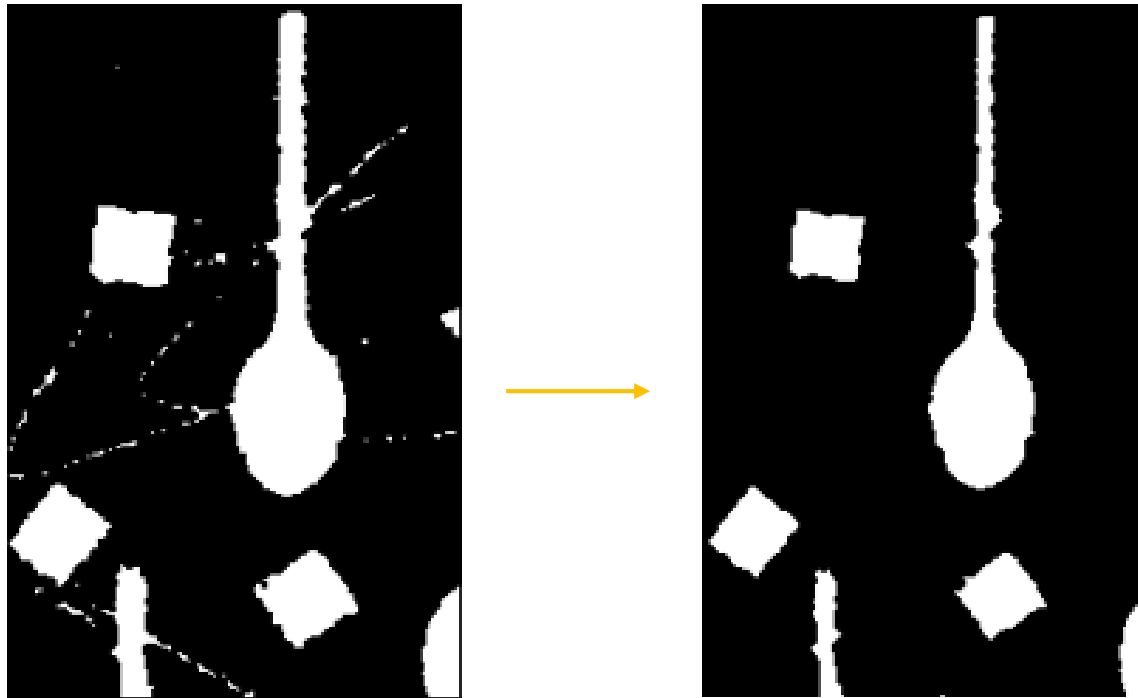
Расширение устраняет шум и способствует объединению областей изображения, которые были разделены шумом, тенями.

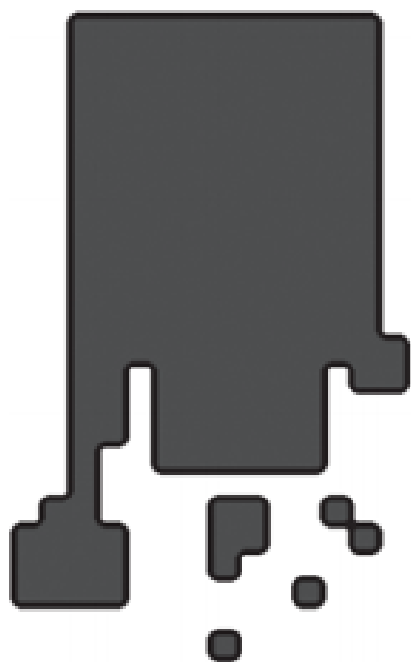


Результат применения дилатации: а – оригинал; б – дилатация; в – после второго применения

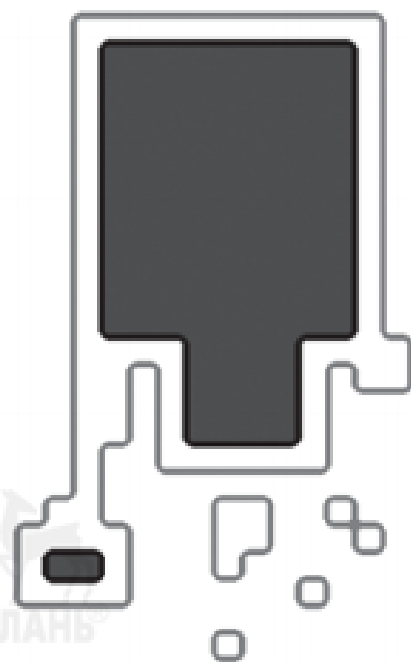
Эрозия

- **Эрозия** (размывание/сужение) изображения обычно используется для избавления от случайных вкраплений на изображении. **Идея состоит в том, что вкрапления при размывании устраниятся, тогда как крупные и соответственно более визуально-значимые регионы остаются.**

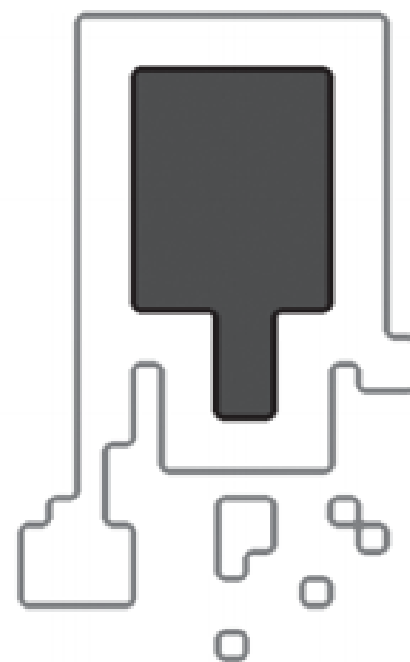




a



б



в

Результат применения эрозии: а – оригинал; б – эрозия; в – после второго применения

- Морфологическое раскрытие (opening)

$$\text{open}(A, B) = (A(-)B)(+)B$$



Сильный шум



Эрозия



Раскрытие

Применение закрытия

- Применим операцию закрытия к изображению с дефектами объектов:



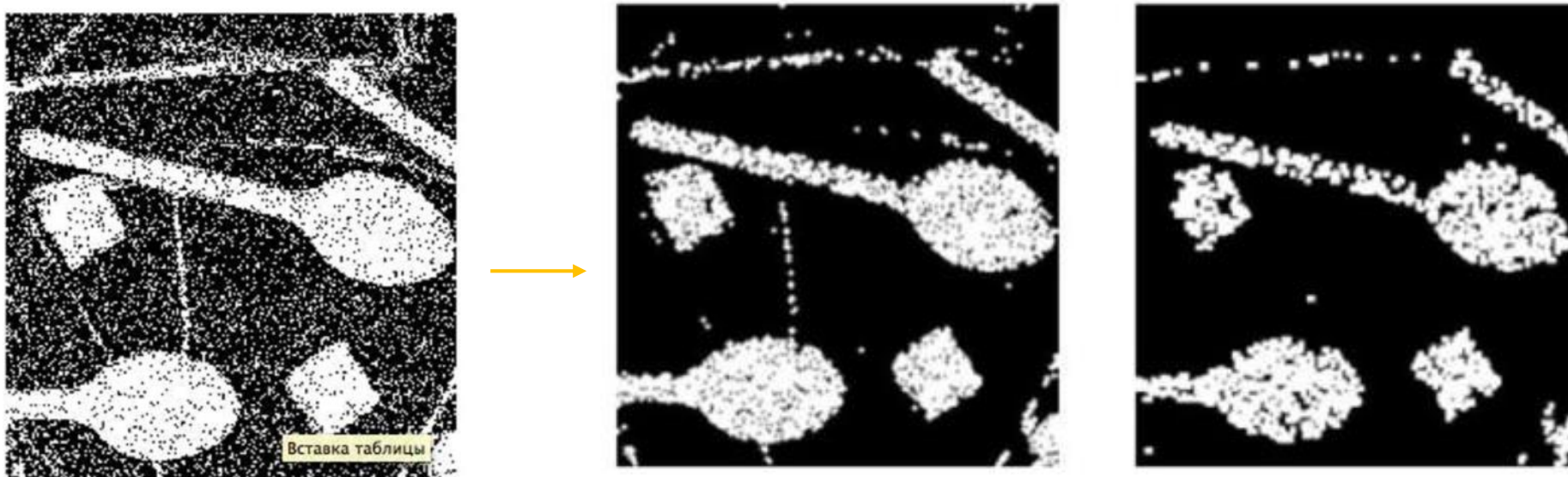
- Морфологическое закрытие (closing)

$$close(A, B) = (A (+) B) (-) B$$

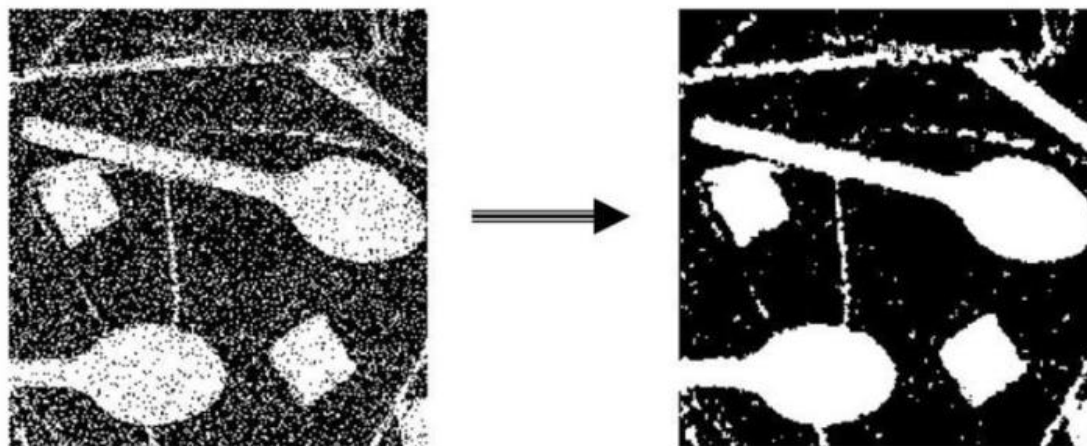
Не лучший пример для морфологии



Применение морфологических операций приведет к таким результатам



В начале желательно применить медианный фильтр



В *OpenCV* эти преобразования реализуются функциями *erode()* и *dilate()*:

```
void erode(  
    const Mat& src,           // входное изображение  
    const Mat& dst,           // выходное изображение  
    const Mat& element,       // ядро, объект типа Mat()  
    Point anchor = Point(-1,-1), // позиция якоря  
    int iterations = 1,       // сколько раз применять  
    int borderType = BORDER_CONSTANT, // экстраполяция границы  
    const Scalar& borderValue = morphologyDefaultBorderValue()  
);
```

Запишем фрагмент кода, применяющий операции эрозии и дилатации к изображению *img*:

```
element = Mat();           // ядро по умолчанию  
erode(img, erodeImg, element); // вычисление эрозии  
dilate(img, dilateImg, element); // вычисление дилатации
```

Операции размыкания и замыкания можно реализовать с помощью многоцелевой функции ***morphologyEx()***, параметры которой аналогичны параметрам функций ***erode()*** и ***dilate()***

```
void morphologyEx(  
    const Mat& src,           // входное изображение  
    const Mat& dst,           // выходное изображение  
    int op,                   // оператор  
    const Mat& element,       // ядро, объект типа Mat()  
    Point anchor = Point(-1,-1), // позиция якоря  
    int iterations = 1,       // сколько раз применять  
    int borderType = BORDER_CONSTANT, // экстраполяция границы  
    const Scalar& borderValue = morphologyDefaultBorderValue()  
);
```

Оператор *op* определяет выбранную операцию: *MOP_OPEN* – размыкание; *MOP_CLOSE* – замыкание; *MOP_GRADIENT* – морфологический градиент; *MOP_TOPHAT* – «Верх шляпы»; *MOP_BLACKHAT* – «Черная шляпа».

Пример использования процедуры дилатации для устранения разрывов



До дилатации

0	1	0
1	1	1
0	1	0

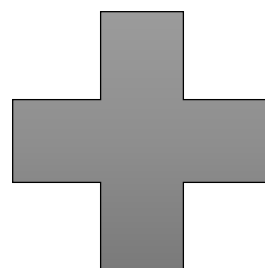
В



После дилатации

0	1	0
1	1	1
0	1	0

В



- Эрозия: $\text{dst}(x,y) = \min_{((x',y') \in [-3..3])} \text{src}(x+x',y+y')$

а а

- Нарращивание: $\text{dst}(x,y) = \max_{((x',y') \in [-3..3])} \text{src}(x+x',y+y')$

а а

- Размыкание: $\text{dst} = \text{open}(\text{src}) = \text{dilate}(\text{erode}(\text{src}))$

а а а

- Замыкание: $\text{dst} = \text{close}(\text{src}) = \text{erode}(\text{dilate}(\text{src}))$

а а а